Open Evidential Tool Bus (O-ETB) v1.1.3 User Manual

Version 10
10 March 2026
Draft

Public Distribution

The Open Group

Contents

1 Introduction **1**

2 O-ETB Assurance Tool configurations **2**

2.1 O-ETB Git repository organisation . . . 2

2.1.1 Development directories . . . 3

2.1.1.1 src directory . . . 3

2.1.1.2 docs directory . . . 3

2.1.1.3 BUILD directory . . . 3

2.1.1.4 TEST directory . . . 4

2.1.2 Database directories . . . 4

2.1.2.1 Knowledge Base (KB) directory . . . 4

2.1.2.2 REPOSITORY directory . . . 4

2.1.2.3 Certification Assurance Package (CAP) directory . . . 4

2.1.3 Assurance outlines, case and evidence repositories . . . 5

2.1.4 Certification Assurance Package . . . 5

2.2 'etb' Assurance Tool . . . 5

2.2.1 Command line options . . . 5

2.2.2 Batch mode command . . . 5

2.2.3 Assurance Tool interactive commands . . . 6

2.2.4 Command Interpreter Syntactic Conventions . . . 9

2.2.5 Command procedures and scripts . . . 11

2.2.6 Timing Commands . . . 11

2.2.7 Certification projects . . . 11

2.3 Assurance case outlines . . . 11

2.3.1 ACO and its one-to-one correspondence to GSN . . . 12

2.3.2 Language overview and authoring principles . . . 12

2.3.3 Explicit relations . . . 12

2.3.4 ACO Workbench . . . 12

2.4 Assurance case patterns . . . 13

2.4.1 Syntax of the pattern language . . . 13

2.4.2 Parameterized elements in the pattern language . . . 14

2.4.2.1 Pattern Parameters 14
2.4.2.2 Extensions 14
2.4.3 Shared Goals and Argument Reuse 14
2.4.3.1 What Does Sharing Mean? 15
2.4.3.2 When Should I Use a Shared Goal Reference? 15
2.4.3.3 What Is a Reuse Key? 15
2.4.3.4 Discovering Sharing Opportunities 15
2.4.3.5 Important Distinctions 15
2.4.3.6 Practical Advice 15

3 Installation, Build and Deployment 16
3.1 Overview 16
3.2 Prerequisites 16
3.2.1 Install prerequisite software 16
3.2.2 Install the O-ETB source files and/or executables 17
3.3 Building and Running the ‘etb’ assurance tool 17
3.3.1 Compile the ‘etb’ policy tool 17
3.3.1.1 Compiling and running within Prolog 17
3.3.1.2 Creating a new ETB executable with make 18
3.3.2 Test the installed ‘etb’ tool 18
3.3.3 Running the examples 19
3.4 Extending the Assurance Tool 19
3.5 Assurance Tool Deployment for MedSecurance 20
3.6 Application to Use Cases and Building the Knowledge Base 20

4 Principles of Operation 21
4.1 Architecture 21
4.2 O-ETB components 21
4.2.1 Functional components 21
4.2.1.1 Core Workflow Engine 21
4.2.1.2 Assurance Case Outline (ACO) Workbench 21
4.2.1.3 Interactive Assurance Tool 21
4.2.1.4 Background Assurance Server 21
4.2.1.5 Tool Agent(s) 21

4.2.1.6 CAP Renderer . . . 22
4.2.1.7 KB Import . . . 22
4.2.2 Data structures and storage components . . . 22
4.2.2.1 Knowledge Base . . . 22
4.2.2.2 Assurance OUTLINES Repository . . . 22
4.2.2.3 Assurance CASES Repository . . . 22
4.2.2.4 EVIDENCE Repository . . . 22
4.2.2.5 Certification Assurance Package . . . 22
4.2.2.6 O-ETB Log . . . 22
4.3 Component interactions . . . 22
4.4 Operating modes . . . 22
4.5 Argument pattern instantiation . . . 24
4.6 Identifier generation and naming during pattern instantiation . . . 26
4.6.1 Unit-scoped identifiers . . . 26
4.6.2 Instantiation and scope lifting . . . 27
4.6.3 Pattern instantiation occurrence . . . 27
4.6.4 Construction of instantiated identifiers . . . 27
4.6.5 Properties of the scheme . . . 27
4.6.6 Relationship to the instantiation workflow . . . 28
4.6.7 User-visible effect . . . 28
4.7 Knowledge Base . . . 28
4.8 Assurance Cases – CASES repository . . . 28
4.8.1 Assurance case directories . . . 28
4.8.2 Persistent assurance case storage . . . 28
4.8.3 Working assurance case cache . . . 30
4.9 Assurance Evidence . . . 30
4.9.1 Evidence Validation . . . 30
4.9.2 Evidence artefacts . . . 30
4.9.3 Evidence categories . . . 31
4.9.4 Evidential tool agents . . . 31
4.9.5 Evidence artefact validation status . . . 32
4.9.6 Re-synchronisation . . . 32
4.10 EVIDENCE repository . . . 33

4.10.1 Evidence category directories . . . 33
4.10.2 Persistent evidence storage . . . 33
4.10.3 Working evidence cache . . . 33
4.11 OUTLINES repository . . . 33
4.12 Certification Assurance Package (CAP) . . . 33
4.12.1 Tailoring the CAP to certification requirements . . . 33
4.12.2 Organising multiple certification projects . . . 34
4.12.3 Text file artefacts . . . 34
4.12.4 HTML directory artefacts . . . 34

5 Assurance Tool Examples 35
5.1 Example 1 – O-ETB Tool instantiating a simple pattern . . . 35
5.2 Example 2 – Assurance Tool exporting an assurance case . . . 36
5.3 Example 3 – Assurance Tool loading a model and instantiating a pattern list . . . 39

6 References 40

7 Abbreviations 42

8 Annex A: O-ETB Configuration Identification 43
8.1 File identification . . . 43
8.2 Module identification . . . 45

9 Annex B: The O-ETB Knowledge Base 48
9.1 Populating the O-ETB knowledge base . . . 48
9.1.1 Approach . . . 48
9.1.2 Stage I – initial UC adoption of MS methodology/tools . . . 48
9.1.2.1 Prerequisites . . . 48
9.1.2.2 Activities . . . 49
9.1.2.3 Results . . . 49
9.1.3 Stage II – UC assurance automation for maintenance and certification . . . 49
9.1.3.1 Prerequisites . . . 49
9.1.3.2 Activities . . . 50
9.1.3.3 Results . . . 50
9.1.4 Stage III – dissemination of the approach & ongoing O-ETB development . . . 50

9.1.4.1 Prerequisites . . . 50
9.1.4.2 Activities . . . 50
9.1.4.3 Results . . . 50
9.1.5 Summary . . . 51
9.2 Model information . . . 51
9.2.1 Architectural model . . . 51
9.2.2 Properties . . . 52
9.3 Argument Patterns . . . 52
9.4 Workflow information . . . 53
9.4.1 Tool flows . . . 53
9.4.2 Workflow programs . . . 53
9.5 Certification requirements . . . 53
9.5.1 Content/process requirements . . . 53
9.5.2 Goal-based requirements . . . 53
9.5.3 Presentation requirements . . . 53
9.6 Evidence-related knowledge . . . 53
9.6.1 Evidence categories and artefact kinds . . . 53
9.6.2 Evidential tools . . . 54
9.7 General knowledge . . . 55

10 Annex C: O-ETB Testing 56
10.1 Functional Tests . . . 56
10.2 Volume and Load Tests . . . 56
10.3 Performance and endurance tests . . . 56
10.4 HTTP API correctness tests . . . 57

11 Annex D: Pattern Instantiation Example 59

12 Annex E: Assurance Glossary 63

List of Figures

1 O-ETB Git repository top-level structure . . . 3
2 Database structure . . . 4
3 O-ETB Functional architecture . . . 21

4 O-ETB component interaction sequence 23
5 O-ETB Tool Agent integration of external tools 23
6 O-ETB major modes and actions 24
7 Argument pattern grammar and rewrite rules 25
8 Example: unit-scoped ACO identifiers lifted to case scope during pattern instantiation 29
9 Tailoring the CAP to certification requirements 34
10 Argument pattern instantiation for a person 38

List of Tables

1 Table Assurance Tool general purpose commands 6
2 Table Assurance Tool etb-mode data and control commands 7
3 Table 3 Assurance Tool aco-mode assurance case outline (ACO) commands NEW . . 9
4 Table Example 1 – instantiating a simple pattern and exporting the result 35
5 Table 5 Running the person_inst procedure 36
6 Table Example 2 – exporting an assurance case as HTML 37
7 Table 7 Running the person_exp procedure 37
8 Table Example 3 – instantiating a pattern list from a model and exporting as HTML . 39
9 Table Abbreviations 39
10 Table 10 O-ETB v1.1.3 directory tree 42
11 Table 11 O-ETB v1.1.3 source code modules 44
12 Table Evidence artefacts and categories 45
13 Table Pattern instantiation correctness tests 54
14 Table Running the system_inst procedure 57

1 Introduction

This User Manual version 10 for the Open Evidential Tool Bus (O-ETB) Version 1.1.3 represents revisions and additions of material actively ongoing. It is a living document with its own version number, as multiple versions of the User Manual may apply to the same version of O-ETB as identified in the title. Higher numbered versions of the User Manual supersede lower numbered versions.

ACKNOWLEDGMENTS:

The O-ETB is based on the Adaptive MILS Evidential Tool Bus (AM-ETB) from the EC CITADEL Project (Critical Infrastructure Protection Using Adaptive MILS, Project Number 700665), implemented by Marius Bozga of Université Grenoble Alpes, with reorganization and continuing development by Rance DeLong of The Open Group. Contributions to argument patterns and assurance tools and approaches were made by atsec information systems, University of York, Fondazione Bruno Kessler, Université Grenoble Alpes and other partners in the CITADEL and D-MILS EC projects.

2 O-ETB Assurance Tool configurations

The O-ETB Assurance Tool may be delivered in several configurations, representing different modes of access to common underlying functionalities, that provide for the construction of assurance cases, along with their supporting evidence, in a form appropriate for evaluation by other parties such as certifiers.

The current configuration of the Assurance Tool provides a command-based interface that can be used interactively and/or in a batch mode to execute single commands or to run pre-defined command scripts or user defined command scripts.

A possible future Assurance Server configuration would provide programmatic access to the core functions of the Assurance Tool through an API. Though not currently implemented, this manual contains comments concerning potential features and properties of such a server.

All configurations run on a shared common file structure defined by O-ETB, or, for development, testing or experimentation, on independent instances of the same file structure, as determined by configuration options and invocation arguments. As part of that file structure, assurance cases and evidence are constructed and updated incrementally and stored persistently in the CASES and EVIDENCE Repositories respectively.

The core workflow of O-ETB utilises *tool agents* to incorporate external evidence-generating tools into the workflow and to store their results in a canonical form into the EVIDENCE Repository.

Tool agents serve as adaptors to include external evidence-producing tools into the O-ETB core workflow and their results into the EVIDENCE Repository. A tool agent embodies knowledge about the data that is required for successful running of a tool, about executing the tool, about interpreting the results of executing the tool, and about reporting the results back to the ETB core workflow through a protocol that includes a combination of tool agent return codes and direct manipulation of files in the EVIDENCE Repository.

O-ETB includes rendering functions that use information from the Repository to create collections of artefacts for certification efforts according to requirements of a particular target certification domain.

2.1 O-ETB Git repository organisation

The O-ETB Git repository structure, available on GitHub at MedSecurance/Assurance and tog-rtd/O-ETB, consolidates development directories and database directories. Development directores contain source code, documentation and other artefacts relating to design, development, testing and deployment. Database directories are relevant to configuraiton and runtime operation.

source code with new directory structure elements for dynamic data resources including a knowledge base, repositories for changeable persistent data comprising assurance cases and assurance evidence, artefact collections comprising certification assurance packages, build space, and documents. Figure 1 shows the top-level structure of the O-ETB source repository. This includes subdirectories related to both development, including src, doc, BUILD and TEST, and subdirectories the hold the runtime and persistent data structures, including KB, REPOSITORY and CAP. Annex A contains a complete list of files in the O-ETB source repository.

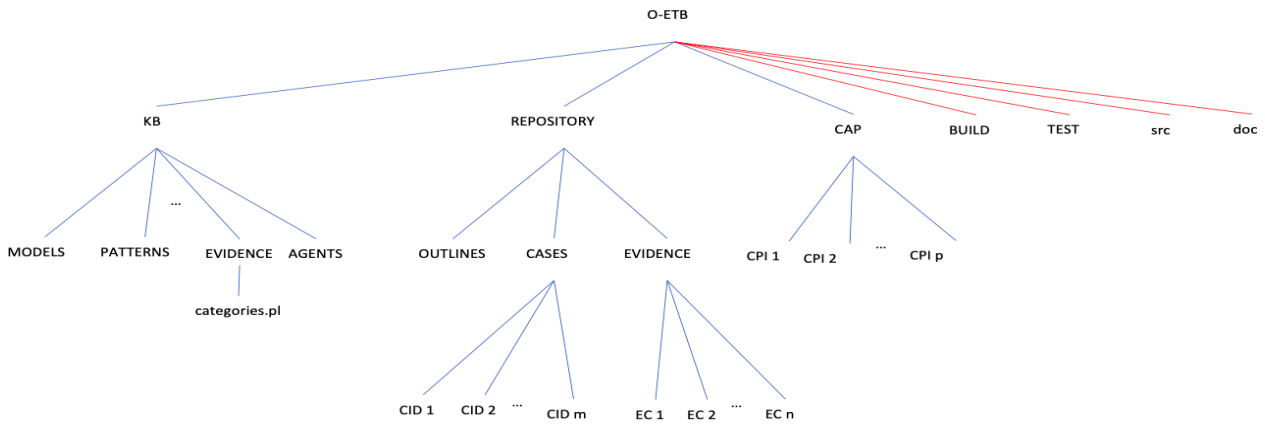


Figure 1: O-ETB Git repository top-level structure

2.1.1 Development directories

These directories contain the implementation source code, tests, build and deployment workspace and documentation for O-ETB.

2.1.1.1 src directory The Prolog source code, including declaration of static and dynamic predicates, has been consolidated into a "src" sub-directory of the O-ETB source repository. This directory and subdirectories contain the source code modules comprising O-ETB.

In the present evolution module files have been added for functionalities such as a command line interpreter, command-invoked actions, command procedures and scripts, self-tests, and parameterization of the implementation.

The source code organization has been enhanced by moving user modifiable source configuration data to the Knowledge Base and by the addition of a global parameters file ("param.pl") that consolidates O-ETB operational settings and constant definitions in one place from where the parameters are referenced at appropriate places in the source code.

Annex A presents the complete list of source code files and modules in O-ETB v1.1.3.

2.1.1.2 docs directory This directory contains README files as well as this User Manual, Release Note, Assurance Glossary (described further in Annex E), argument patterns document, Assurance Case Outline (ACO) specification, ACO Quick Reference Card and design documents and diagrams.

2.1.1.3 BUILD directory The BUILD directory contains the etb and etb-server executables built by the makefile residing in the src directory. It also contains other files related to build and deployment of O-ETB. Since the etb executable references the TEST, KB, REPOSITORY and CAP directories with a prefix of '../', etb should be run either from BUILD (if running the pro-compiled version) or from the src directory (if loading and running from source).

2.1.1.4 TEST directory This directory contains O-ETB test cases, and in the future will contain more test cases, regression test scripts, and test results.

2.1.2 Database directories

These directories contain the data structures that represent the persistent but mutable state of an O-ETB database instance. These contain target-of-assurance information and derived artefacts for potentially multiple assurance and certification efforts that share a common (but dynamic) knowledge base.

Figure 2 illustrates the structure of the database portion of the O-ETB source repository.

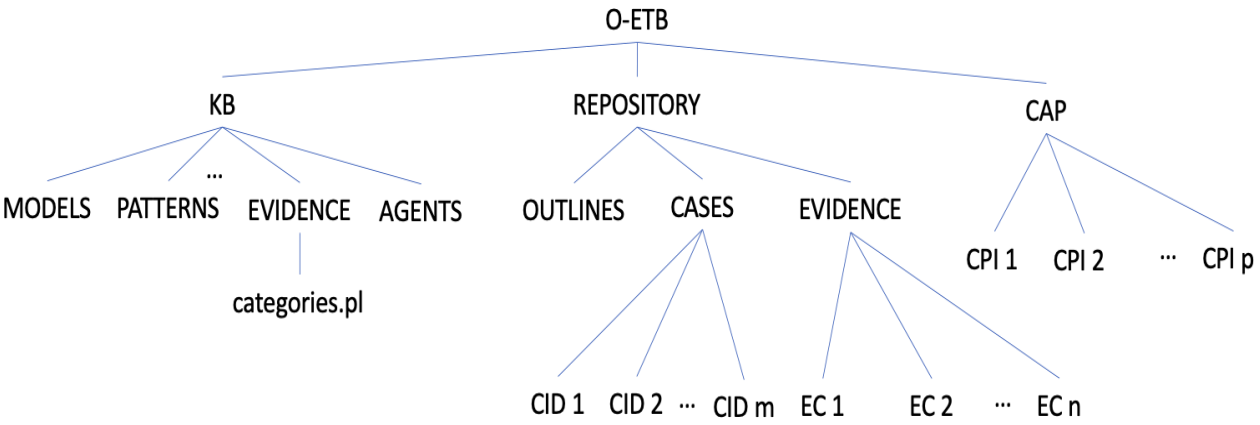


Figure 2: Database structure

2.1.2.1 Knowledge Base (KB) directory A distinct place in the runtime file structure has been created for the knowledge base used by O-ETB ("KB" directory). The knowledge base includes the library of assurance case argument patterns ("PATTERNS" sub-directory), the library of system models ("MODELS" sub-directory), a library of workflow patterns ("WORKFLOWS" sub-directory), and in the future will be extended with other directories for additional expert knowledge applicable to the construction of assurance cases, evidence categories with corresponding evidential tools, and specifications for the requirements on the content and presentation for certification assurance packages.

2.1.2.2 REPOSITORY directory The repository is where O-ETB stores internal representations of artefacts comprising aspects of assurance cases under construction. Specifically, there are branches for assurance case outlines (OUTLINES), instantiated assurance cases (CASES) and evidence records and status (EVIDENCE).

2.1.2.3 Certification Assurance Package (CAP) directory The CAP directory is the place to which O-ETB exports assurance cases from the repository into forms suitable for browsing the results and making results available for further processing.

2.1.3 Assurance outlines, case and evidence repositories

Under the “REPOSITORY” directory the assurance outlines repository (“OUTLINES”), the assurance case repository (“CASES” sub-directory) and the evidence repository (“EVIDENCE” sub-directory) are located. Separate directories are created under CASES for each assurance case. The EVIDENCE directory is organized by directories corresponding to evidence categories and under those are sub-directories are used for individual evidence records.

2.1.4 Certification Assurance Package

The runtime file structure includes a default location (“CAP” directory) for the export of collections of artefacts tailored to specific certification targets.

2.2 ‘etb’ Assurance Tool

The ‘etb’ tool enables AC pattern development, AC pattern instantiation and Assurance Tool development and testing. The etb tool provides the ability to test AC patterns under development and use of the knowledge base and tool agents.

2.2.1 Command line options

When a compiled version of the O-ETB Assurance Tool is started from the command line as “etb”, several command line options are recognized. NEW

- --command, -c <etb command> execute the specified command and exit. Note that this command could be proc or script, which will run a previously defined sequence of commands NEW
- --model, -m <model directory> load a model specification from the specified directory under KB/MODELS before the --command, -c option is processed if it has been specified or before entering the interactive command interpreter. NEW
- --patterns, -p <patterns file name> load a user-provided patterns definition file. This option is processed before the --command, -c option, or if that option has not been specified then before entering the interactive command interpreter. NEW
- --procs, -x <procs file name> load a user-provided procs definition file. This option is processed before the --command, -c option, or if that option has not been specified then before entering the interactive command interpreter. NEW
- --test, -t run self-tests when the tool is started before entering the interactive command interpreter
- --verbose, -v show maximum detail for all messages when available

2.2.2 Batch mode command

If the ‘etb’ Assurance Tool is started with the --command or -c option it executes the specified command in *batch mode* and O-ETB exits. In addition to a single command, the specified command may cause the execution of a command procedure or script when it is the proc or script command

respectively. Procs that are used for examples, demos and system maintenance are predefined in the O-ETB source code (in the files src/procs.pl for general procs and procs_etb.pl for ETB-specific procs). Command procedures may also be defined in an arbitrary user-provided file containing one or more user-defined procs. Such a procs file may be loaded using the --procs option or the interactive load_procs command described in the following section. The user may also use the script command to run in batch mode a script supplied by the user in a separate file (one script per file).

O-ETB performs other options, specifically the --model , --patterns and --procs options, before running the batch command. The O-ETB exits after running the batch mode command.

2.2.3 Assurance Tool interactive commands

If the --command or -c option is not specified the Assurance Tool enters its top-level command interpreter and offers the prompt "etb>" on the console. There are basic general commands available in the normal mode (user) and an extended set of commands for use by a developer in development mode (advanced).

The command interpreter is extensible. The command interpretation machinery and the generic commands are defined in the command module. Commands for specific groups of functionalities can be defined in separate smaller files where the syntax, argument types, operational semantics and help text for such commands can be specified. Commands that invoke ETB-specific functions are currently defined in the file command_etb.pl. It is anticipated that a set of commands for knowledge base or evidence management may be defined separately in the future. Entering the command "help" will list all the commands available in the current operating mode.

Table 1 shows the general purpose commands available in the Assurance Tool.

Table 1: Table Assurance Tool general purpose commands

advanced. | basic. | developer. | etb.	Switch to advanced (basic, developer or etb) mode.
halt.	Exit the etb tool. (Will also terminate a server spawned from an interactive instance.)
help.	List the commands available in the current mode.
help(<*command name*>).	Give a synopsis of the named command.
load_procs(<*file*>). NEW	Load user-created proc definitions from an arbitrary user-specified file. Proc names must be unique globally.
proc(<*proc name*> [, step]).	Run the named stored procedure, optionally pausing after each command.
proc(<*proc name*> [, verbose]).	Run the named stored procedure, optionally verbose.

advanced. | basic. | developer. | etb.	Switch to advanced (basic, developer or etb) mode.
quit.	Terminate etb command loop or script, but stay in Prolog top level. Only used for development activity.
regtest.	Run built-in regression tests.
script(<*file name*> [, step]).	Run the named command file, optionally pausing after each command.
script(<*file name*> [, verbose]).	Run the named command file, optionally verbose.
selftest.	Run built-in self tests.
show_proc(<*proc name*>). NEW	Output a pretty-printed listing of the specified proc.
show_procs. NEW	Output a compact listing of all currently defined procs, including those loaded by the user.
time(<*etb command*>).	Execute <*etb command*> and report time stats.
time(<*etb command*>, <*reps*>).	Execute <*etb command*> the number of times indicated by <*reps*> and report total time stats.
traceoff.	Turn off tracing.
traceon.	Turn on tracing.
traceone.	Turn on tracing for only the next command.
version.	Display the current version number and version description.
versions.	Display past and current version numbers with descriptions.

Table 2 shows the data and control commands available in the ETB Assurance Tool ‘etb-mode’.

Table 2: Table Assurance Tool etb-mode data and control commands

attach_case(<*assurance case id*>).	Attach the identified assurance case in the CASE repository.
attach_evidence.	Attach the EVIDENCE repository.
detach_case.	Detach the currently attached assurance case.
etb_reset.	Perform reset of ETB databases including dynamically created persistent objects.

attach_case(<*assurance case id*>).	Attach the identified assurance case in the CASE repository.
etbt.	Run currently pre-configured default ETB test.
etbt(<*test id*>).	Run the identified ETB test.
etbt(<*test id*>, <*mode*>).	Run the identified test in <*mode*>.
etbtests.	Run all ETB tests.
export_case(<*base name*>, <*format*>).	Export the current assurance case to named sub-directory in the CAP directory.
import(<*spec type*>, <*file*>, <*sid*>).	Import the <*spec type*> specification from <*file*> and store in the KB as <*sid*>.
instantiate_pattern(<*pattern name*>, <*pattern arguments list*>, <*assurance case id*>).	Instantiate a pattern from the KB into the CASES repository with the given case id.
instantiate_pattern_list(<*list of pattern_name-arguments_list pairs*>, <*assurance case id*>).	Instantiate a list of patterns from the KB into the CASE repository with the given case id.
load_model_v(<*model id*>, <*policy var*>, <*platform var*>, <*config var*>).	Load system model from the KB and instantiate the provided variables. Used in command scripts.
show_case(<*case id*>). NEW	Display the specified case from the CASES repository.
show_cases. NEW	Display all the cases in the CASES repository.
show_pattern(<*pattern name*>). NEW show_pattern(<*pattern name*>,<*format*>).	Display the named argument pattern. Format is one of: {pp, text, header }. Pp is pretty print, text is dense unformatted text, header is only the pattern name and arguments. Unspecified format is assumed to be pp.
show_patterns. NEW show_patterns(<*format*>). show_pats.	Display all defined patterns. Format is one of: {pp, text, header } as above. Unspecified format is assumed to be text. The show_pats command uses header format.
update.	Update CASES and EVIDENCE to incorporate asynchronously updated evidence.

Table 3 shows the assurance case outline (ACO) commands available in the Assurance Tool 'aco-mode'.

Table 3: Table 3 Assurance Tool aco-mode assurance case outline (ACO) commands NEW

aco_apl(<*aco file*>, <*apl file*>).	Translate an ACO file into the corresponding APL representation for subsequent instantiation and CAP generation.
aco_aplc(<*aco file*>, <*canon apl file*>).	Same as aco_apl but the aco file is first canonicalized
aco_canon(<*aco file*>, <*canon aco file*>).	Canonicalize IDs from an aco file to a new aco file
aco_check(<*aco file*>).	Perform semantic checks on an ACO file (identifier scope, explicit-relation well-formedness, and structural consistency) and report issues.
aco_parse(<*aco file*>).	Parse an ACO file and report any syntax/structural errors detected by the ACO processor.
aco_stats(<*aco file*>).	Compute summary statistics for the outline, including node counts by type, undeveloped elements, and explicit-relation diagnostics.
aco_tree(<*aco file*>). aco_tree(<*aco file*>, <*options*>).	Render an ASCII tree view of the ACO outline, showing the indentation-derived argument structure. Options are: full, structure and skeleton.

It is expected that additional advanced commands for development and diagnostics may be added in the future. The extensibility of the command interpreter facilitates this.

2.2.4 Command Interpreter Syntactic Conventions

The Assurance Tool's command interpreter defines the commands accepted by the Tool. The set of commands is flexible and extensible, with the syntax, argument semantics, help texts and operational semantics defined in one of two files. General commands are defined in the file src/com/command.pl and ETB-specific commands are defined in the file src/command_etb.pl. More information about extending the command set may be found in Section 2.5.

For convenience of implementation, the Assurance Tool utilizes the Prolog reader to read interactive commands from the console and command scripts from files. Each command conforms to the simple syntax of a Prolog term, which is either an atom, a number, a variable, or a compound term consisting of an atomic name followed by a sequence of argument terms within a pair of parentheses. Each

interactive command entry is terminated by a period. Commands that are part of a sequence of commands in a script or a proc are separated by commas.

It is useful to be aware of some lexical rules concerned with Prolog textual input. If these rules are observed, the syntactic interaction with the Assurance Tool will be straightforward. Any input read by the Prolog reader will treat unquoted tokens that start with an upper-case letter followed by alphanumeric characters or the character "_" as a *logical variable*. Similarly, unquoted alphanumeric tokens that start with a lower-case letter are treated as an *atom*. Any string of characters in single quotes is also treated as an atom. Therefore, the token "Word" is a logical variable with the variable name "Word". The token "word" is an atom, and the single-quoted token " 'Word' " is also an atom. The Assurance Tool user will not typically employ logical variables and therefore as a general rule will want to write atoms. By adopting the practice of always using atoms adorned with single quotes the troubles incurred by inadvertently using a logical variable rather than an atom can be averted. If this best practice is followed the final comments below on the matter may be ignored.

The user can save some typing by recognizing that any atom that starts with a lower-case letter need not be enclosed in single quotes. Caution is advised, though, since tokens starting with a lower-case letter but which is followed by other characters that are considered separators in the Prolog term syntax can cause the recognition of an atom not be as expected. An unadorned atom token starting with a lower-case letter can only contain, as subsequent characters, other upper- and lower-case characters, digits and the special symbol "_" (underscore). If a token starts with an underscore it is treated as a logical variable. If an atom is to contain any other special characters, such as "-" (dash) or other punctuation or Prolog separators, it must be single-quoted or else the dash or other special character will be interpreted as standing for or beginning a separate token. Virtually any sequence of characters can be used as an atom if the sequence is enclosed in single quotes. However, escape characters have special effect, and if they are to be included in a single- or double-quoted sequence of characters they must be escaped or their special effect will be invoked.

Examples of terms:

Atoms: a z1 'a' 'z1' 'A' 'Z1' (the first & third are the same atom, the second and fourth are the same atom, the first and fifth are distinct atoms, as are the second and sixth)

Numbers: 1 7 21 More complex numbers, such as floats and rationals, are recognized but not typically used in O-ETB commands.

Logical Variables: A Z1 (the variable A and the atom 'A' are distinct and the variable Z1 and the atom 'Z1' are distinct) Logical variables should NOT be used in O-ETB commands *except* as they may be used in procs or scripts, but *never* in commands given in the interactive interpreter mode. This is because the scope of a logical variable in a proc or script is the entire proc or script and all instances of the variable name in that proc or script refer to the same variable. However, in the case of an interactive command the scope of the variable is limited to that one command. Even if the same variable name is used in another interactive command it refers to a distinct new logical variable. Procs and scripts are described in the following section.

Lists: [x, y, z] [2, a, d] ['Name1', 'Name2']

Compound Terms: foo(bar) g(b, 8, [x, z]) The number of arguments is known as the arity of the term. The first example has arity 1 and the second has arity 3. An atom is a 0-arity term.

2.2.5 Command procedures and scripts

There are predefined command procedures ("procs") that run the examples and can be used for testing and demonstrations. At the "etb>" prompt a predefined procedure (e.g. named "myproc") can be run with the command proc(myproc). It can be run with verbose output with the command proc(myproc,verbose). It can be made to prompt and wait for user instruction to proceed (empty line input) with the command proc(myproc,step).

The file src/com/procs.pl defines system-wide predefined procedures not relating to O-ETB-specific data or operations. The procedures utilise the same commands available at the command prompt. The O-ETB developer may define additional procedures in the procs.pl file for subsequent execution as above. The base set of procs defined in procs.pl is extended by procs related to O-ETB-specific data, operations or O-ETB examples. These are defined in the file src/procs_etb.pl. User-defined procs, unless adopted as a utility to be included with the distribution, should be defined in user-created files which can then be specified in a load_procs command.

A sequence of 'etb' commands can also be stored in a file, in which case it is referred to as a *script*. Scripts may be run with a script command, analogous to the proc command, with the script file name substituted for the stored procedure name. verbose and step are valid options also for the execution of scripts. User-defined scripts should be created in files containing one script per file

2.2.6 Timing Commands

The 'etb' Assurance Tool has two commands for timing execution of other 'etb' commands. The first, time(etb_command) times the execution of a single command. The second, time(etb_command, repetitions) executes the command for the number of times specified by the repetitions argument, and reports timing statistics. The second version turns off console output from the repeated execution of the command to eliminate the substantial timing impact of generating console output, leaving a result that is more representative of the processing time that would be consumed to do the equivalent work load.

2.2.7 Certification projects

The collection of artefacts needed for a certification are known to O-ETB as a named *certification project*, which is created and maintained under the CAP directory. It represents the current rendering, for a target certification effort, of relevant content from the CASES and EVIDENCE Repositories. It will typically grow and evolve over many sessions of assurance case and evidence development and evolution. The registration of distinct certification projects is not yet completely implemented in O-ETB. The export procedures currently export to the CAP directory, not a project subdirectory in the CAP.

2.3 Assurance case outlines

Assurance Case Outlines (ACO) provide a textual, line-oriented representation of assurance cases with a strict one-to-one correspondence to the Goal Structuring Notation (GSN) community standard.

Each ACO construct directly maps to a GSN element, preserving argument intent while enabling version-controlled, text-based authoring suitable for automation.

2.3.1 ACO and its one-to-one correspondence to GSN

2.3.2 Language overview and authoring principles

2.3.3 Explicit relations

In addition to indentation-derived structure, ACO supports explicit natural-language relation sentences. These are used to express author-intended relationships that cannot be represented purely by a tree structure. Explicit relations are recognized only in accepted sentence forms and only when all referenced identifiers have been declared within the same compilation unit (MedSecurance Project, 2025a).

An ACO file is written as an outline. Indentation is significant and expresses the primary argument structure. In the default semantics, indentation-derived relationships determine supported-by (decomposition) and in-context-of (context attachment) edges. Body text lines provide rationale, explanatory material, and supporting narrative.

ACO is designed for incremental development. Early outlines may contain placeholders (e.g., undeveloped modules or pending evidence) that are later refined. The Workbench and CLI tools provide immediate feedback through structural rendering and statistics.

> Assurance Case Outline (ACO) is a textual notation for authoring assurance arguments that maintains a deliberate one-to-one correspondence with the industry-standard Goal Structuring Notation (GSN). Each ACO node header denotes a specific GSN element: Goals, Strategies, Contexts, Assumptions, Justifications, and Evidence. The ACO processor preserves this correspondence so that authors and reviewers can reason about the outline exactly as they would about the equivalent GSN argument.
>
> The normative definition of ACO syntax and semantics is provided by the ACO v1.2 Specification (MedSecurance Project, 2025a). A compact, author-facing summary is provided by the ACO v1.2 Reference Card (MedSecurance Project, 2025b).

2.3.4 ACO Workbench

Diagnostics and statistics are computed over explicit author-written relations and over indentation-derived tree edges separately. This prevents accidental keyword matches in body text from producing illegitimate internal relation structures, and ensures that cross-branch statistics reflect only explicit author intent.

The Assurance Case Outline (ACO) subsystem provides a GSN-aligned language and associated processors for authoring and analyzing assurance case outlines. The subsystem includes: (i) the ACO language definition, (ii) the ACO processor (parser and validator), (iii) command-line interfaces in

both aco and etb> modes, and (iv) the ACO Workbench, which supports interactive development.

The Workbench supports structural rendering (ASCII tree views), assessment and statistics (including undeveloped elements, module references, and explicit relation diagnostics), and transformation operations on outlines. Transformations are designed to preserve author intent while supporting canonicalization and subsequent translation to APL for instantiation and CAP generation (MedSecurance Project, 2025a).

2.4 Assurance case patterns

An *assurance case pattern* is a representation in an assurance case pattern language (e.g., [ ]) containing typed logical variables that can be instantiated producing a concrete term that is immediately translatable into an *assurance case fragment* in whatever representation is being used for assurance cases (such as the Goal Structuring Notation (GSN) [ ] or Structured Assurance Case Metamodel (SACM) [ ]).

The assurance case pattern language has a well-defined grammar that represents a structure of argumentation, evidence, and metadata that includes typed logical variables that can be instantiated resulting in another grammatically valid sentential form that may still include logical variables or a ground sentence representing an assurance case or part of an assurance case. A special type of logical variable in an AC pattern is of *Evidence* type (or a collection of *Evidence* types corresponding to the different classes of evidence produced by different evidential tools or processes carried out by persons). Such variables become instantiated to terms that are referred to as *evidence specifiers*.

Evidence specifiers are treated like other ground terms in an assurance case fragment even though they may still contain variables the instantiations of which are deferred until the running of the evidential tools/processes that produce the ground instances of the evidence data. The running of such tools/processes is done by the O-ETB when it is asked to produce a concrete assurance case or to update an existing assurance case because of some change that may cause the values of the produced evidence data to change.

In summary, the O-ETB constructs assurance cases by executing workflows and instantiating assurance case patterns stored in its knowledge base (KB). The operation of the O-ETB can be tailored by supplying suitable workflows and assurance case patterns in the KB.[1]

A textual language representation is easily constructed from a graphical representation of a GSN assurance case and vice versa. The present implementation of the language does not yet include some of the standard GSN structural abstraction extensions for AC patterns but similar extensions are part of the currently proposed new requirements for O-ETB.

2.4.1 Syntax of the pattern language

Top level of assurance case pattern

[1]Currently, workflows are not distinct from assurance case patterns. The patterns embody the assurance strategies and workflows. Workflows here refers to a potential extension of O-ETB to include an explicit workflow coordination language.

<ac pattern> ::= ac_pattern(<pattern id>, <formal parameters>, <goal>)

<formal parameters> is a list of parameter names

<goal> ::= goal(<id>, <claim>, <context>, <body>)

<claim> ::= <textual atom>

<context> ::=

<body> ::=

2.4.2 Parameterized elements in the pattern language

An assurance case pattern may declare formal parameters that occur within the body of the pattern. When the pattern is being instantiated, actual parameter values provided are substituted for the formal parameters in the instantiated assurance case.

2.4.2.1 Pattern Parameters Pattern formal parameters may be declared as lists and may be used in places within the pattern where lists might normally appear, such as in an iterator. The actual parameter value provided when the instantiation is invoked must be an actual list constant.

2.4.2.2 Extensions The GSN Community Standard introduces a graphical notation for these features but this requirement suggests a textual representation that is consistent with the graphical notation and considered adequate for O-ETB in the context of MedSecurance. To that end, O-ETB will allow the following constructs in the target of GSN relations:

- alternative(<list of alternative elements>) – *optionality* – one or more (inclusive OR)
- alternative (<list of alternative elements>, <min>, <max>) – *optionality* – m of n, e.g. 1 of 3 (exclusive OR)
- optional(<element>) – *optionality* – 0 or 1 occurrence of the instantiated <element>
- multiple(<element>) – *multiplicity* – 1 or more occurrences of the instantiated <element>
- multiple (<element>, <cardinality>) – *multiplicity* – definite number of instantiated occurrences of <element>
- multiple (<element> , <low>, <high>) – *multiplicity* – indefinite number of instantiated occurrences of <element> ranging from <low> to <high>

These abstractions are applicable to all GSN relation types (i.e., SupportedBy and InContextOf).

There will also need to be some operations on "AC forms" (ACs that are at any stage of instantiation from uninstantiated pattern up to fully instantiated AC). Such operations include determining whether there are still uninstantiated terms, or finding uninstantiated terms, etc.

2.4.3 Shared Goals and Argument Reuse

In large assurance cases, the same argument may be relied upon in more than one place. O-ETB supports this through `ac_shared_goal_ref`, which allows authors to explicitly state that an argument is shared.

2.4.3.1 What Does Sharing Mean? Sharing means that:

> The same assurance claim, supported by the same justification and evidence, is intentionally relied upon in more than one place in the case.

Using a shared goal reference makes this intent explicit and avoids accidental duplication.

2.4.3.2 When Should I Use a Shared Goal Reference? You should use `ac_shared_goal_ref` when:

- The claim being made is the same in each location (not merely similar).
- The same body of evidence supports the claim everywhere it is used.
- The claim remains valid in each parent context without modification.

If any of these conditions do not hold, the argument should not be shared.

2.4.3.3 What Is a Reuse Key? To share an argument, you must identify a *reuse key*. The reuse key is the minimal set of properties that must be identical for the claim and its evidence to remain valid. Common examples include:

- A software component together with its specification and version.
- A certification together with its certificate identifier.
- A qualified tool or process together with its qualification scope and version.
- A platform configuration together with its profile and build identifier.

Parent contexts may differ in other respects, provided all assumptions of the shared argument are satisfied.

2.4.3.4 Discovering Sharing Opportunities Sharing opportunities are often identified during review rather than during initial authoring. O-ETB supports converting existing argument subtrees into shared goals after the fact. Tool support may assist by identifying similar claims or structures, but the final decision to share is an explicit authoring choice.

2.4.3.5 Important Distinctions

- A shared goal reference expresses identity, not just dependency.
- Dependency relations such as `supports` explain how arguments relate, but do not imply sharing.
- Sharing implies that changes to the shared argument or its evidence affect all places where it is used.

2.4.3.6 Practical Advice If you would be comfortable pointing multiple reviewers to the same evidence package to justify a claim, then that claim is a good candidate for sharing.

3 Installation, Build and Deployment

3.1 Overview

The executable O-ETB system is composed of two physical configuration items, the 'etb' Assurance Tool and the 'etb-server' Assurance Server. The Assurance Tool is used to develop and assurance case patterns and develop assurance cases. The ETB Assurance Tool and Server use those patterns to create instantiated assurance cases. These components share a common set of Prolog source modules and their separate executables are constructed using the make command with a simple ***makefile***, which accompanies the sources. A primary objective of the 'etb'/'etb-server' development is to create a lightweight and highly portable evidential tool bus framework that can be easily adapted to different situations and applications, that has a minimum of external dependencies, and that requires minimal resources to run. This objective has been achieved to a high degree in the current implementation.

3.2 Prerequisites

The 'etb' Assurance Tool and the 'etb-server' Assurance Server are implemented in the Prolog language and require the SWI-Prolog environment to run. This is the software's primary external dependency apart from the operating system and a small number of utilities. The version of SWI-Prolog used for the current version is version 8.2.4 available from www.swi-prolog.org.

SWI-Prolog is available for several operating environments, including Microsoft Windows (64 bit) and (32 bit), MacOS X 10.6 and later on Intel, and several Linux versions including Ubuntu. It is also available in Docker containers and as a source distribution that one can build locally.

Our O-ETB implementation uses *only* the libraries that come with the Prolog distribution or small open source utility packages that are included in the O-ETB repository. Furthermore, the functional architecture has been organized so as to place adaptation into the hands of application developers without requiring modification to the core implementation. Rather the implementation is adaptable through the contents of its O-ETB Knowledge Base.

The software is provided as a set of Prolog source files and/or as "executable" files that have the Prolog runtime environment already linked in. Prior to installing and building the software it is required that Prolog be installed. The exception to this requirement is if there is already a compiled version of 'etb' and 'etb-server' for the target platform, in which case all the dependencies are already linked into the executable.

3.2.1 Install prerequisite software

Install the following:

- SWI Prolog – Prolog language system and libraries, version 8.4.2 or later. SWI Prolog is available for several operating environments, including MacOS, Windows, and Linux. See (`http://www.swi-prolog.org`)

- gsn2x – rendering a textual representation of GSN as a graphic image – not yet used but expected to be used (https://github.com/jonasthewolf/gsn2x)
- graphviz / dot – graphical rendering of input provided in the textual dot notation (https://graphviz.org/download/)
- make – standard Unix environment build tool
- find – standard Unix environment tool

The environment must be configured so that the Prolog runtime can execute the other external software items listed above using "shell", "exec" or "forkexec".

3.2.2 Install the O-ETB source files and/or executables

The current version of the etb tool consists of a directory tree, presented in Figure 1 and Figure 2, including source files and example files. The complete configuration list is given in Section 8 Annex A. The distribution is provided as a zip file of this directory tree, in the project internal repository at MedSecurance/Internal/WPs/WP3/AssuranceWork/O-ETB-version-1.1.0, or as a GitHub repository.

3.3 Building and Running the 'etb' assurance tool

The etb assurance tool is implemented in Prolog and requires the SWI Prolog environment to run. The etb tool can be provided as a set of Prolog source files and/or as an "executable" that has the Prolog runtime environment already bundled in. This executable is made by the makefile located with the source files that must be run in an environment that has SWI Prolog installed.

3.3.1 Compile the 'etb' policy tool

If a ready-made executable 'etb' has been provided it may be executed directly from a command shell prompt (e.g. from the src directory, ../BUILD/etb) resulting in the ETB prompt "etb>". If this is the case skip down to the next section. Otherwise (or if the provided executable is out-of-date) take the following steps:

3.3.1.1 Compiling and running within Prolog The ETB tool may be compiled and run manually within the Prolog interactive top-level command loop. This is the a good approach if you are not using a prepackaged deployment. Do as follows:

To compile manually, start SWI-Prolog from a command shell prompt using the name of the SWI-Prolog executable (usually 'swipl', 'swi-pl', or something similar, depending on how SWI-Prolog was installed).

After printing a short banner SWI-Prolog will display its prompt "?- ".

At the Prolog prompt enter "[etb]." (without the quotes but with the square brackets and the period). (Note that this is different to entering "etb." at the Prolog prompt, which executes, rather than compiles, 'etb'. Entering the name "etb" within brackets tells Prolog to consult the file "etb.pl" which is the

top-level file of the source. This results in compiling etb.pl and all of the modules and included files that it references.

Prolog will compile the code and print “true.”

Now execute the compiled code by entering at the Prolog prompt: “etb.”

Now you should see the ETB prompt “etb>”.

One benefit of this approach is primarily for a developer or tester, that is, the ability to break out of the ETB command interpreter to the Prolog prompt to inspect data structures, to perform operations not provided as ETB commands, to use the interactive Prolog debugger, and to incrementally edit and recompile the code without affecting (mostly) the current data structures. The ETB command interpreter may be re-joined without complete reinitialization by entering the command ‘etb’ at the Prolog prompt (“?- “). For convenience, the command “make”, which is a Prolog-level command, is also provided as an ETB-level command so it is not necessary to exit the ETB command interpreter to incrementally recompile the running ETB. Another benefit is that the command history within the ETB command loop works by running ETB this way, allowing one to scroll back, line edit and re-execute previous commands.

3.3.1.2 Creating a new ETB executable with make In the source directory “src” execute the “make” command. It will use the provided makefile. This will (re-)create the ETB executable in the ../BUILD directory (../BUILD/etb). Executing the newly built executable, as for a ready-made executable, will result in the prompt “etb>”. The makefile also contains the targets clean, clean_cap and clean_repo. These can be used (as in “make clean”) to restore the CAP, the REPOSITORY or both to a fresh empty state. It is useful to do this frequently while testing. The “clean” make targets can be used whether the makefile or the manual method is used to compile and run the Tool. The “make clean” target can also be run from within the ETB command loop with the command “etb-reset”.

Note that a compiled ETB executable may be invoked with the command line options detailed in Section 2.2.1. It is not necessary to use any of these options for the present purpose of demonstration.

3.3.2 Test the installed ‘etb’ tool

The O-ETB Assurance Tool has some built-in self tests. These may be run to ensure that everything is working correctly after building ‘etb’. Follow the instructions to run the ‘etb’ tool. If you compiled manually in Prolog start it with the Prolog prompt command “etb(self_test).” This will run some built-in self-tests when it starts. Other alternatives to self_test are regression_test, no_initial, and verbose. The lists of self tests and regression tests are defined in the file param.pl. The no_initial option, which bypasses much of the initialization, and verbose are useful primarily to ETB developers. To not run any of these options simply start the tool with the Prolog prompt command “etb.”

The self-tests can also be run by starting ‘etb’ normally and entering at the ‘etb’ prompt the command “selftest.”

Procedures made up of ‘etb’ commands may be predefined in the procs.pl file. Look at the ones there and try them by entering the etb command “proc(ProcName).”, where ProcName is the name of one of the procedures defined in procs.pl.

3.3.3 Running the examples

There are several examples included with the sources of the O-ETB Policy Tool. These include examples described in documents and PowerPoint slide decks used to introduce the O-ETB concepts.

There are predefined procedures ("procs") that run the examples. At the etb> prompt a predefined procedure (e.g. named "myproc") can be run with the command proc(<*procname*>). It can be run with verbose output with the command proc(<*procname*>,verbose).

It is instructive to examine the file procs.pl that defines the predefined procedures. Each procedure definition consists of a list of the same etb commands available at the command prompt contained within the structure proc(<*procname*>,[<*command*>, <*command*>, . . .]). The commands are given within the list without the terminating "." that is required when they are given interactively. However, each proc definition does end in ".".

See Section 5 for further details on the currently included examples. The user may define additional procedures in the procs.pl file for subsequent execution as above.

3.4 Extending the Assurance Tool

The most common extension to the tool is to enable the integration of new external evidential tools into the O-ETB core workflow by creating an appropriate *tool agent*.

The 'etb' tool can be easily extended in several ways:

- Commands can be added by modifying the command module to add a syntax, semantics (optional), help, and do clause for the new command. A syntax clause must be added for the command. This clause declares the command name and parameters, and what mode the command belongs to, admin or advanced. Admin commands are available in admin mode, but also accessible in the advanced mode but not vice versa.
- The self-test framework is implemented in the test module. Tests for specific new modules can be added in the TEST subdirectory. An example of a test definition file for the etb module is implemented in TEST/etb_test.pl.
- New predefined 'etb' command procedures can be added to the procs module. A proc clause is added for each new procedure to be defined. There are examples in the procs.pl file.

Global parameters are set in the param module. Settable parameters (those that can be changed from the 'etb' command line with the set command) are itemized in a list settable_params. For example, the parameter audit_logging is a settable parameter. Adding new settable parameters requires the new parameter name to be added to the list of settable parameters and to the dynamic directive above it in a fashion similar to the other entries.

Extending the 'etb-server' can be extended in several ways:

- Adding APIs
- Adding increased parallelism

Such server extensions tend to be more involved that just adding new commands or tests although adding APIs can be done my mimicking the structure of the code for existing APIs.

3.5 Assurance Tool Deployment for MedSecurance

Specifics of deployment in the MedSecurance environment is described elsewhere..

3.6 Application to Use Cases and Building the Knowledge Base

Section 3.3.4 of the MedSecurance deliverable D3.1, Automated IoMT Security Analysis & Assurance Toolbox – Interim Version, describes the approach and strategy for developing the knowledge base (KB) of the Assurance Tool through its initial use on the project use cases (UCs). The goal is to build a KB that will enable the O-ETB to construct an assurance case (AC) and a certification assurance package (CAP) in a manner that is efficient, consistent, repeatable, preserves acquired knowledge, and memorializes the details of processes previously completed successfully.

The strategy comprises three stages:

Stage I – the initial stage consisting of manual interactive use, in which the models, properties, AC patterns, and certification requirements are developed and encoded in the KB. More about populating the KB is contained in Section 9 Annex B.

Stage II – the automation stage in which the models, properties, AC patterns and certification evidence requirements are used with the workflow automation and pattern instantiation provided to refresh the structure of the AC and re-generate certification artefacts when changes are made to system specifications, assurance methods or certification requirements. Using the AC patterns and models developed for the project UCs the AC will be re-instantiated and evidence re-generated/validated. This phase may be accompanied by some efforts to generalize the AC patterns library and other KB contents. The Assurance Server API and/or command scripts will be used to enable the greatest extent of automation possible.

Stage III – the post project stage in which the Assurance Tool, Assurance Server and the KB developed during the project, including efforts to generalize those results are applied, augmented and refined in future projects, and further enhancements are made to the Assurance Tool and Assurance Server implementation.

Refer to Section 3.3.4 of the MedSecurance D3.1 for a detailed description of the prerequisites, activities and results of each of the Stages introduced above.[2]

[2] D3.1 has confidentiality “Partners Only Distribution”. Use case partners have access to D3.1. For other non-project general users the information from Section 3.3.4 will be replicated here in a future revision of this User Manual.

4 Principles of Operation

4.1 Architecture

In this section we refer to the O-ETB functional architecture shown in Figure 3.

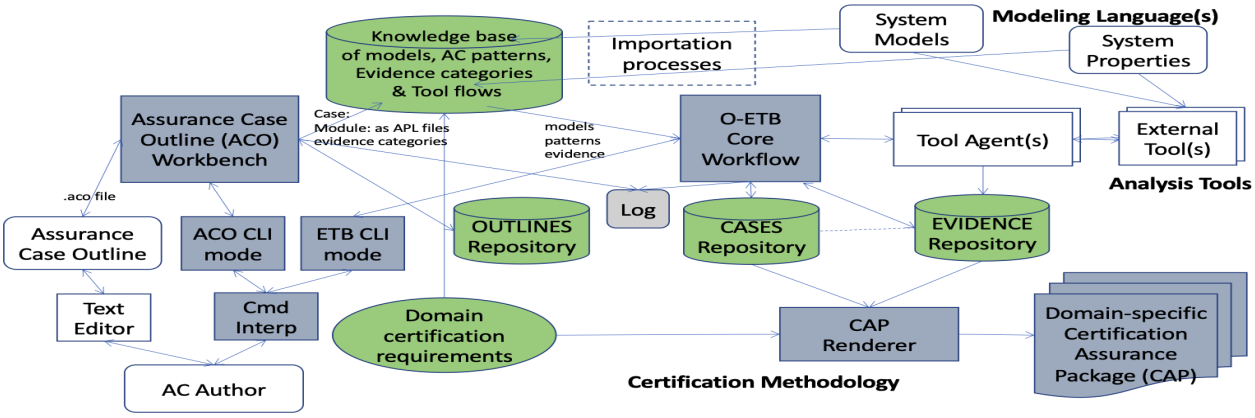


Figure 3: O-ETB Functional architecture

4.2 O-ETB components

4.2.1 Functional components

4.2.1.1 Core Workflow Engine Includes functions that generate instantiated assurance case patterns in the CASES Repository, creates evidence records needed to make the assurance case valid in the EVIDENCE Repository, invokes external evidential tools to validate evidence and to update evidence status in the EVIDENCE repository

4.2.1.2 Assurance Case Outline (ACO) Workbench The ACO Workbench provides parsing, rendering, analysis, and transformation capabilities for assurance case outlines. It supports tree visualization, statistical analysis, consistency checking, and controlled transformations to internal O-ETB representations.

4.2.1.3 Interactive Assurance Tool The O-ETB has an interactive ‘etb>’ command mode and an ‘aco>’ command mode.

4.2.1.4 Background Assurance Server The server is not currently implemented.

4.2.1.5 Tool Agent(s) Invocation of executable wrappers for external evidential tools to interface them to O-ETB core workflow and the EVIDENCE Repository.

4.2.1.6 CAP Renderer Functions to export assurance cases and evidence artefacts in a form to satisfy specific certification evidence presentation requirements.

4.2.1.7 KB Import Though not yet implemented, automated procedures are foreseen for converting external artefacts into KB format artefacts, such as system architectural models and properties of components.

4.2.2 Data structures and storage components

These constitute mutable but persistent objects that are the intermediate or final results of O-ETB processing that survive executions sessions.

4.2.2.1 Knowledge Base Contains several categories of files containing assurance case argument patterns, workflows, system models and properties, evidence category-to-tool mappings, certification artefact requirements, etc.

4.2.2.2 Assurance OUTLINES Repository

4.2.2.3 Assurance CASES Repository Persistent storage of dynamic assurance case elements.

4.2.2.4 EVIDENCE Repository Persistent storage of evidence records with status and evidential tool results.

4.2.2.5 Certification Assurance Package

4.2.2.6 O-ETB Log

4.3 Component interactions

Interaction among O-ETB components is illustrated in the sequence diagram in Figure **??**.

Figure 4 Component interaction sequence

Figure 5 illustrates the integration of external tools by tool agents.

4.4 Operating modes

A summary of the major modes and actions is shown in Figure 6.

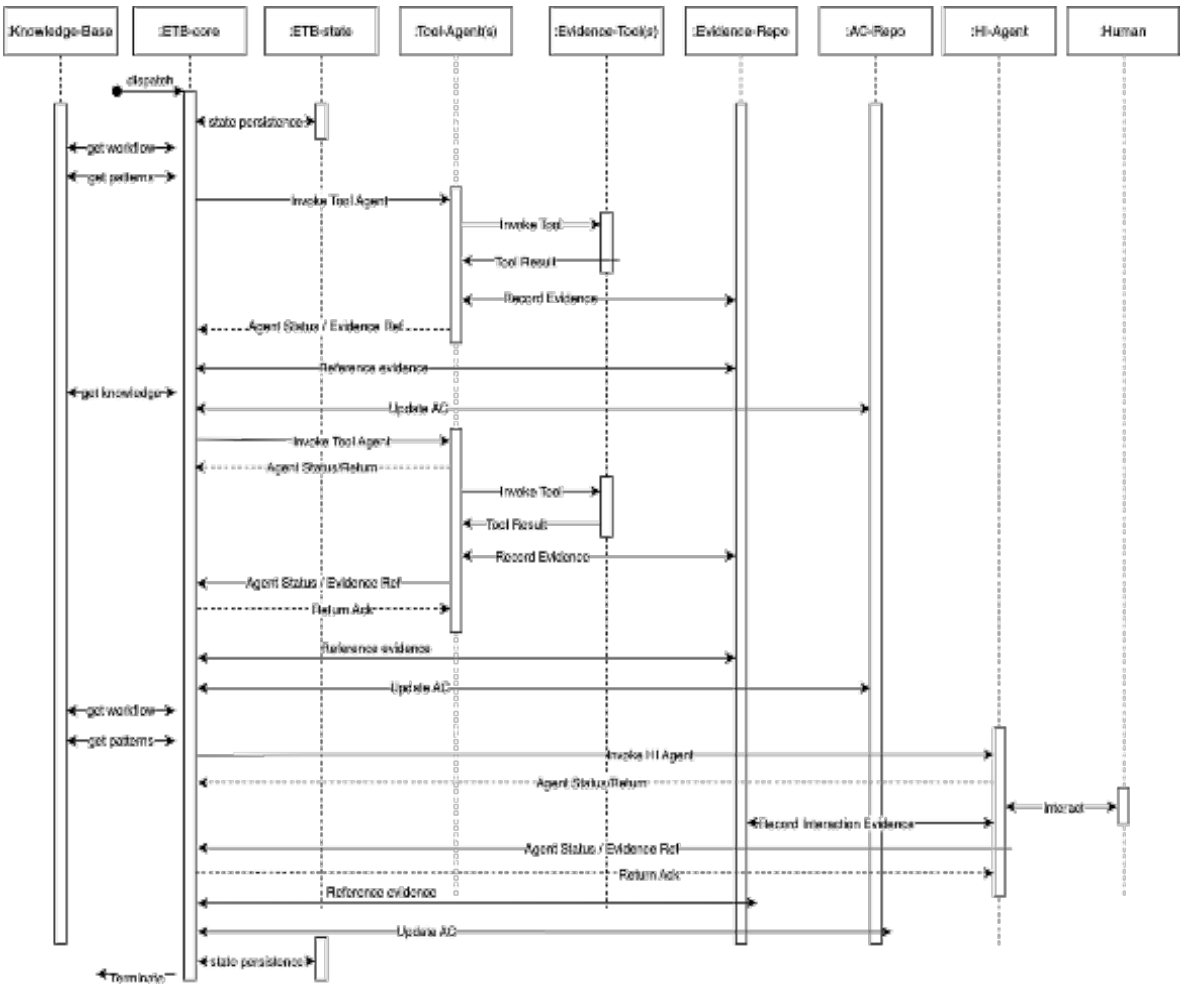


Figure 4: O-ETB component interaction sequence

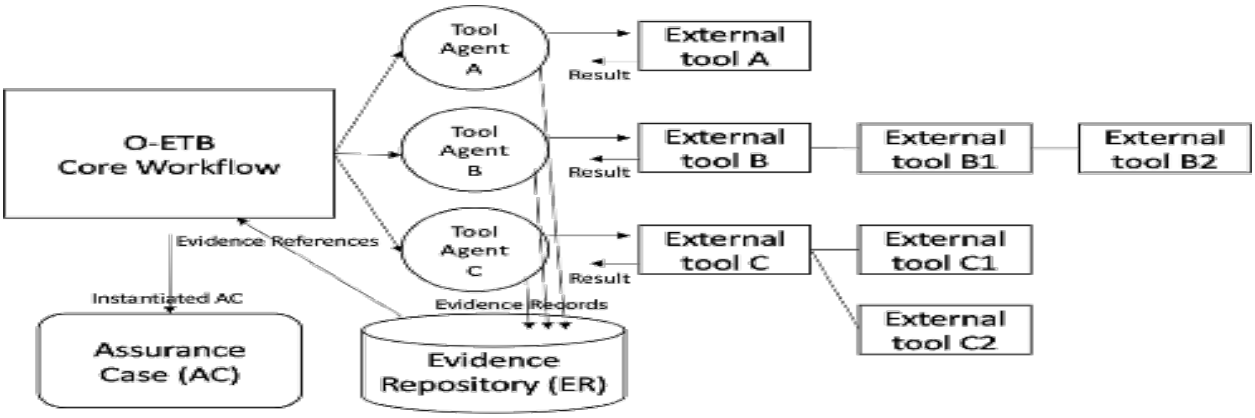


Figure 5: O-ETB Tool Agent integration of external tools

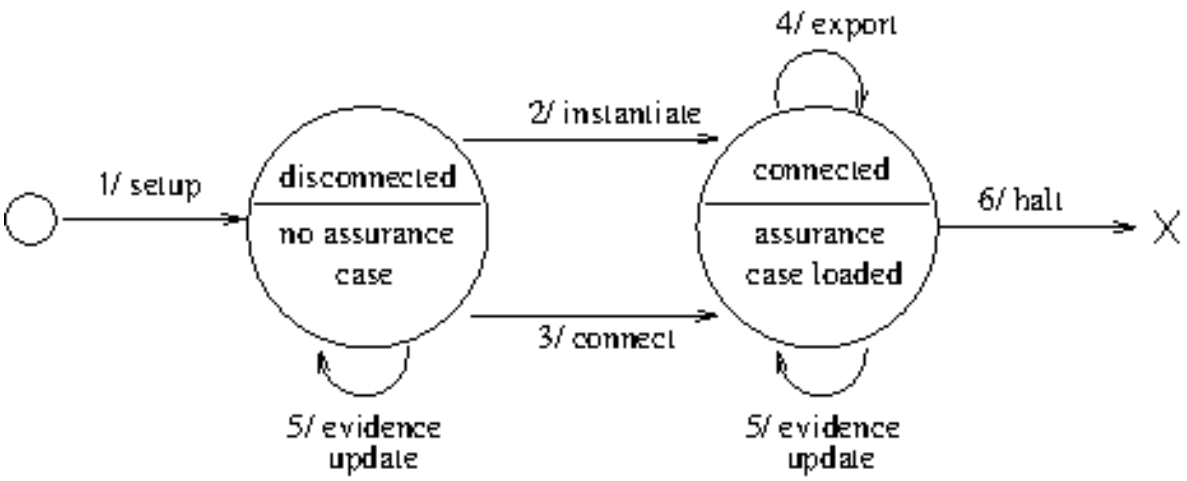


Figure 6: O-ETB major modes and actions

4.5 Argument pattern instantiation

O-ETB workflow is driven primarily by assurance case argument pattern instantiation. Construction of the argumentation and the generation of the associated supporting evidence is accomplished by the instantiation of a chosen top-level pattern using information from a specified system model.

Figure 7 provides the definition of the pattern instantiation rewrite rules.

The following notational conventions are used:

- *A* => *C* is a rewrite rule where *A* represents the sentential form to be rewritten and *C* the resultant sentential form.
- $$[[Z]]_{M\mu}$$

  denotes the instantiation of an assurance case element Z with respect to the model M and using the mapping of formal parameters μ.
- Underlined symbols (e.g. <u>context</u>) are terminal symbols of the pattern language.
- Symbols with an initial capital letter (e.g. *Claim*) are logical variables that stand for a term, the type of which is intended to be inferred from the intuitive connotation of the ordinary meaning of the capitalized symbol.
- Bold symbols (e.g. **eval** or **if then else**) represent operational semantics.
- An *Iterator* is one of:
  - <u>iterate</u>(*Name*, *Category*, <u>list</u>(*Values*))
  - <u>iterate</u>(*Name*, *Category*, <u>expand</u>(*ActArgName*))
  - <u>iterate</u>(*Name*, *Category*, <u>subjects</u>(*PolicyArgName*))
  - <u>iterate</u>(*Name*, *Category*, <u>ss_flows</u>(*PolicyArgName*))
  - <u>iterate</u>(*Name*, *Category*, <u>nodes</u>(*PolicyArgName*))
  - <u>iterate</u>(*Name*, *Category*, <u>modes</u>(*Subject*))
  - <u>iterate</u>(*Name*, *Category*, <u>transitions</u>(*Subject*))

$[\ \underline{\text{pattern}}(\ Id, Args, Goal\ )\ ]_{\mathcal{M},\mu} \Rightarrow [\ Goal\ ]_{\mathcal{M},\mu/Args}$

$[\ \underline{\text{goal}}(Id, Claim, [\ Context, \ldots\ ], [Support, \ldots]\ ]_{\mathcal{M},\mu} \Rightarrow$
$\underline{\text{goal}}(Id\#\mu,\ [\ Claim\ ]_{\mathcal{M},\mu}, [\ [\ Context\ ]_{\mathcal{M},\mu}, \ldots\ ], [\ [\ Support\ ]_{\mathcal{M},\mu}, \ldots])$

$[\ \underline{\text{context}}(Claim)\ ]_{\mathcal{M},\mu} \Rightarrow \underline{\text{context}}(\ [\ Claim\ ]_{\mathcal{M},\mu}\ )$

$[\ \underline{\text{assumption}}(Claim)\ ]_{\mathcal{M},\mu} \Rightarrow \underline{\text{assumption}}(\ [\ Claim\ ]_{\mathcal{M},\mu}\ )$

$[\ \underline{\text{justification}}(Claim)\ ]_{\mathcal{M},\mu} \Rightarrow \underline{\text{justification}}(\ [\ Claim\ ]_{\mathcal{M},\mu}\ )$

$[\ \underline{\text{strategy}}(Claim, [Context, \ldots], [Goal, \ldots])\ ]_{\mathcal{M},\mu} \Rightarrow$
$\underline{\text{strategy}}(\ [\ Claim\ ]_{\mathcal{M},\mu}, [\ [\ Context\ ]_{\mathcal{M},\mu}, \ldots], [\ [\ Goal\ ]_{\mathcal{M},\mu}, \ldots\ ])$

$[\ \underline{\text{strategy}}(Claim, Iterator, [Context, \ldots], [Goal, \ldots])\ ]_{\mathcal{M},\mu} \Rightarrow$
$\underline{\text{strategy}}(\ [\ Claim\ ]_{\mathcal{M},\mu}, [\ [\ Context\ ]_{\mathcal{M},\mu}, \ldots], [\ \bigcup_{\mu':SArgs \to \mathcal{M}} [\ Goal\ ]_{\mathcal{M},\mu \sqcup \mu'}, \ldots\ ])$

$[\ \underline{\text{evidence}}(Cat, Claim, [\ Context, \ldots]\ )\ ]_{\mathcal{M},\mu} \Rightarrow$
if call Agent@$Category(\mathcal{M}, \mu)$ **==** (**valid**, $xref$) **then**
$\underline{\text{evidence}}(\ [\ Claim\ ]_{\mathcal{M},\mu}, [\ [\ Context\ ], \ldots], xref)$
else $\perp$

$[\ \underline{\text{goal-ref}}(Id)\ ]_{\mathcal{M},\mu} \Rightarrow \underline{\text{goal-ref}}(Id\#\mu)$

$[\ \underline{\text{pattern-ref}}(Id, ActArgs)\ ]_{\mathcal{M},\mu} \Rightarrow \underline{\text{goal-ref}}(\ GId\#\ [\ ActArgs\ ]_{\mathcal{M},\mu}\ )$, **eval** $[\ Pattern\ ]_{\mathcal{M},\mu}$
(where $Pattern$ is the pattern identified by Id and GId is its top goal identifier)

Figure 7: Argument pattern grammar and rewrite rules

When the instantiation procedure commences the top-level pattern given to the instantiation procedure becomes the current sentential form to be instantiated. Instantiation proceeds by left-to-right recursive descent on the structure of the form, triggering specific evidence generating tool agents when evidence (leaves) are encountered during pattern instantiation.

Two cases require a specific treatment beyond the (standard) recursive development:

- The instantiation of an invoked goal (goal-ref) requires the additional instantiation of that goal in the calling context, with the current evaluation of parameters. That is, the instantiation is not limited to the pattern itself but triggers additional instantiation of all other encountered sub-patterns.
- The instantiation of an evidence leaf node requires interaction with the agent associated with the evidence category. When the evidence is valid, it is included in the final result. Otherwise, the assurance case will remain incomplete and the missing evidence is reported to the user.

This presentation of the instantiation procedure as a set of rewrite rules has been kept abstract for the sake of exposition. A number of straightforward extensions are however possible, such as the following, that will be considered as the implementation of the O-ETB core workflow evolves:

- memoization of intermediate instantiation results, in particular, to avoid the costly re-generation of the same piece of evidence, referred many times in the assurance cases,
- versioning of segments of the assurance case to work hand-in-hand with memoization to reduce assur- ance case maintenance recompute overhead,
- concurrent execution, that is, allow instantiation to proceed concurrently for independent sub-goals and/or evidence. Such an option might ensure a faster construction process in particular when non-automatic (human-in-the-loop) and/or compute-intensive verification agents are used to produce and check specific evidences,
- additional fine-grained and expressive workflow notation and control mechanisms i.e., choose alternatives based on metrics, depart from depth-first instantiation, exploiting parallelism, restricting or allo- cating execution for some overall time budget, threshold amount of missing evidence, etc.

4.6 Identifier generation and naming during pattern instantiation

4.6.1 Unit-scoped identifiers

In the Assurance Case Outline (ACO) language, outlines are organized into *units*. A unit is introduced by a *unit header* (for example, `Case:` or `Module:`). Within a unit, the outline consists of *node headers* (for example, `Goal`, `Strategy`, `Context`, `Assumption`, `Justification`, `Evidence`, and `Module` nodes).

Identifiers appearing in node headers are *unit-scoped*: they are required to be unique within the enclosing unit, but they are not required to be globally unique across different units. This design supports modular authoring, reuse, and independent development of assurance case fragments.

4.6.2 Instantiation and scope lifting

When a `Module` unit is instantiated into a concrete assurance case, all node identifiers defined within that unit must be lifted from unit scope into a *case-wide* namespace. This lifting is necessary to ensure that:

- multiple modules can be composed within a single assurance case, and
- the same module can be instantiated multiple times without identifier collisions.

O-ETB performs this lifting during pattern instantiation by allocating a single *instance identifier* for each *pattern instantiation occurrence*.

4.6.3 Pattern instantiation occurrence

A pattern instantiation occurrence corresponds to one concrete expansion of a `Module` reference during assurance case construction. Each occurrence is distinct, even if:

- the same module is instantiated multiple times, or
- the same module is instantiated multiple times with identical actual arguments.

Each occurrence is assigned exactly one instance identifier, which is stable for the duration of the instantiation and recorded in the assurance case repository for traceability.

4.6.4 Construction of instantiated identifiers

For every node instantiated from a given occurrence, O-ETB derives the case-wide identifier by concatenating the original unit-scoped identifier with the instance identifier:

$$GlobalId = LocalId _ InstanceId$$

where:

- *LocalId* is the identifier written by the outline author in the node header (unit-scoped),
- *InstanceId* is the identifier allocated to the instantiation occurrence (case-wide), and
- *GlobalId* is the identifier that appears in the instantiated assurance case and in exported CAP artifacts.

For example, if a module defines a goal with identifier `G10` and that module is instantiated with instance identifier `17`, the instantiated goal identifier becomes `G10_17`.

4.6.5 Properties of the scheme

This identifier generation scheme ensures that:

- identifiers remain readable and retain a clear connection to the author-provided outline,

- no identifier collisions occur in the instantiated assurance case,
- no restrictions are imposed on the syntax of unit-scoped identifiers beyond the ACO language definition (including support for canonical dotted identifiers), and
- all nodes originating from the same pattern instantiation occurrence can be recognized by their shared instance identifier.

4.6.6 Relationship to the instantiation workflow

During assurance case construction, the O-ETB instantiation engine maintains a set of pending and running pattern instantiations. Each pending instantiation is represented internally by a triple consisting of: the referenced module, the bound actual arguments, and the allocated instance identifier. All node identifiers produced while instantiating that occurrence are derived from the same instance identifier, ensuring consistency and traceability across the instantiated case.

4.6.7 User-visible effect

Authors and reviewers will observe the instantiated identifiers (for example, `G1_3`, `S2_3`, `E10_3`) in generated assurance case presentations and CAP exports. These identifiers are tool-generated, deterministic within an instantiation run, and are intended to provide global uniqueness while preserving a clear relationship to the original outline structure.

4.7 Knowledge Base

4.8 Assurance Cases – CASES repository

4.8.1 Assurance case directories

The CASES repository is organised by assurance case identifier.

4.8.2 Persistent assurance case storage

The Assurance Tool and the Assurance Server are designed to build and maintain an assurance case incrementally because there are many distinct assurance activities that contribute to the construction of the assurance case and these activities may be performed over an extended period of time. Intermediate results and even incomplete results must be capable of being captured in one run and to be picked up and used to continue the effort by another run.

O-ETB uses a feature of the Prolog implementation to create, manipulate and store a database of such results and evolving structures. The SWI Prolog library ***persistency*** provides persistent storage for one or more dynamic predicates. O-ETB employs this feature for several persistent stores, including the CASES Repository and EVIDENCE Repository. O-ETB modules that are responsible for these databases declare the terms that can be placed in the database using the directive ***persistent***. By convention and implementation the declaring module is the sole agent having access to the persistent

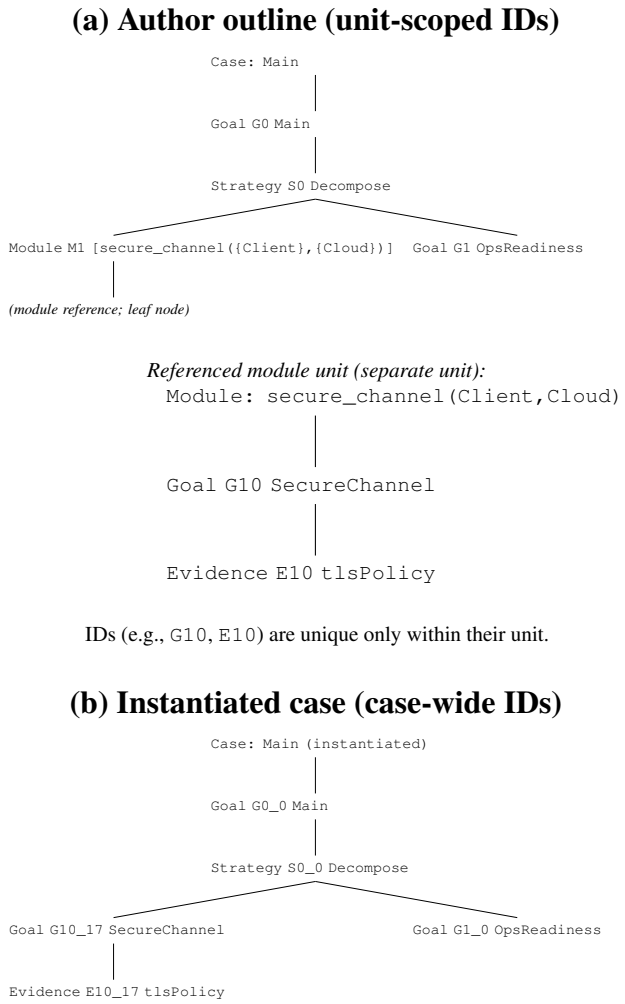


Figure 8: Example: unit-scoped ACO identifiers lifted to case scope during pattern instantiation

database and provides an internal API to access the database from other modules of the O-ETB system. As there may be multiple instances of the module executing in distinct instances of the Tool or Server, some operations require locking of critical sections in the O-ETB implementation.

4.8.3 Working assurance case cache

Each instance of the Assurance Tool or the Assurance Server has a working store for the currently attached assurance case. This cache represents the in-memory copy of the declared persistent predicates and may be synchronized with the persistent CASES Repository, the current assurance case can be detached, and the cache cleared and reloaded with a different assurance case from the Repository.

4.9 Assurance Evidence

Evidence is data that provides information about some development artefact (specification, program code, tests, etc) that supports a goal or claim concerning key properties of the artefact that render it fit for purpose. Properties may include security, safety, privacy, functional correctness, performance, etc. Evidence results from performing some assurance activity and capturing the results to be referenced by argumentation for the related goal or claim. Typical assurance activities include:

- Formal verification that system models (which may include formal specifications and/or source code) satisfy formally specified properties or exhibit a required behaviour when executed (actually or symbolically).
- Verifying, by inspection and testing by independent experts, that an artefact or an entire system conforms to applicable and mandated standards.
- Verifying that the specific operating configuration of a system and its deployment and operating environment are consistent with the constraints and assumptions that have been imposed by the development and implementation.
- Analysis of the system architecture, components, and environment to discover potential faults and failures and to determine that appropriate steps have been taken to prevent or to mitigate the effects of such faults and failures. Such analyses may emphasize safety or security or some other characteristic that is essential for the system.

4.9.1 Evidence Validation

The assurance processes utilised by MedSecurance involve diverse assurance activities that create concrete evidence artefacts. The collection of evidence artefacts, along with the assurance case that organizes, relates, and presents the evidence, serves as a basis for belief that claims made by the assurance case are valid.

4.9.2 Evidence artefacts

An evidence artefact is generated by one of multiple potential sources, such as:

- Inspection by a person to confirm a claim such as conformance by a development artefact to an industry standard or a project guideline. Such an inspection action may be entirely manual or may be assisted by one or more tools employed by the inspector that have not been integrated with the automated assurance process.
- A demonstrated and confirmable history of reliable use of a tool for a purpose similar to that for which the tool is being employed to create or support evidence directly used for the present assurance effort.
- Review by qualified experts to confirm that given facts and observations about the entity being assured support a conclusion that is in agreement with a target claim.
- Execution of a test case or collection of test cases in a particular test scenario and configuration to confirm that an expected result is produced.
- Execution of an analysis tool on one or more development artefacts using a specific set of situation-dependent input arguments to confirm that the tool produces an expected result or result that supports a claim.

4.9.3 Evidence categories

As discussed previously, for each argument pattern claim or sub-claim requiring substantiation one or more categories of evidence are identified that are capable of supplying the needed substantiation. Each evidence category is associated with a method or tool that is used to produce a concrete evidence artifact of the associated category given the inputs defining the current instance.

The results of performing the method or executing the tool are subject to an interpretation to determine that the result does not represent an exception and that its value is valid and sufficient to support the claim to which it is to be applied.

4.9.4 Evidential tool agents

O-ETB constructs assurance cases that are dependent upon external assurance activities and tools that produce the evidence referenced by the assurance case argumentation. ACs are modular in that they may include assurance goals that are not yet shown to be achieved by the existence of valid evidence. A complete and valid subtree of the AC, for which all needed evidence is complete, can serve as evidence for another goal. Only such reasoning, and tautological reasoning, is performed by the O-ETB itself. For all other evidence, it depends on external processes and tools. Each tool provides a certain class of evidence. When an item of evidence is required to make an element of the AC valid the specified class of the evidence is used to choose a tool agent corresponding to that class.

A tool agent manages the interaction of the O-ETB internal workflow with an external evidence-producing tool. Each tool agent communicates to the core workflow engine through a commonly defined interface and with the evidence repository through its own commonly defined interface. The tool agent embodies the interaction protocol with the specific tool or tool cluster that produces the needed class of evidence. The tool agent marshals the needed arguments, invokes the tool, interprets the tool's response and reports to the core workflow after storing the evidence in the evidence repository. The tool agent is responsible determining success of the evidence validation. That is, the agent must determine that the tool output is consistent with the result needed to satisfy the current goal.

The tool agent is required to perform the following actions:

- marshal the inputs and arguments to the tool in a form the conforms to the requirements of the tool.
- invoke the tool with the collected inputs and arguments.
- monitor the execution and/or receive the results of the tool invocation.
- recognize exceptions raised during the execution of the tool and take appropriate action.
- parse the results and interpret them to the extent necessary to determine that they conform to the expected format, that they do not represent an error condition, and that they appear to represent a valid outcome. The validation status is recorded appropriately as 'valid' or 'invalid'.
- store the outcome in an evidence record of the EVIDENCE Repository in a canonical form that has been defined for the corresponding evidence category, and the status is recorded.
- depending on whether the tool agent was launched for synchronous, asynchronous or silent operation return to the O-ETB core workflow, signal completion to the O-ETB server through its API, or silently exit.
- when the validation status of a tool invocation or manual assurance activity cannot definitively be reported as 'valid' or 'invalid' by the tool agent it is reported as 'ongoing'.

For evidence checking that reports a validation status of 'ongoing' the validation activity continues asynchronously with respect to O-ETB (for example as a manual process carried out by people). In such cases, once a definitive verdict of 'valid' or 'invalid' is reached the validation status file is updated either by the asynchronously running tool agent or by the Server's workflow program upon notification through the API.

In some cases, a group of external tools work in concert under the coordination of, and invoked by, a control process. In such cases the tool agent is designed to interface only with the control process and to leave the sequencing and communication with and among the other tools to the control process.

Evidence categories that correspond to assurance activities performed manually by one or more people, whether or not the activity is supported by tools outside of the O-ETB workflow, are included in the O-ETB workflow through special UI agents that deliver the relevant facts to a designated responsible individual who will in turn respond to the UI agent or programmatically invoke the Server API to report results of the assigned activity.

4.9.5 Evidence artefact validation status

The previous paragraph alludes to the validation status of an evidence artefact. The validation status is contained within a status file that is created within the EVIDENCE Repository when the artefact is initially created. Upon creation the status is recorded as 'pending'.

4.9.6 Re-synchronisation

EVIDENCE Repository updates by asynchronous tool agents do not immediately result in update to assurance cases that are supported by the evidence artefacts. Rather, update of assurance cases occurs when manually initiated by the user of the Assurance Tool or by the Assurance Server either periodically or when prompted through an API request.

4.10 EVIDENCE repository

4.10.1 Evidence category directories

Evidence records of a particular evidence category are stored in a subdirectory named for the category.

4.10.2 Persistent evidence storage

A Prolog feature for persistent data bases is used for the evidence storage. The Prolog program manipulates a copy of the data in memory which is automatically synced to a file in the file system for persistence.

4.10.3 Working evidence cache

The running Assurance Tool or Assurance Server maintains a local working cache of evidence records that are backed by the persistent EVIDENCE Repository.

4.11 OUTLINES repository

The OUTLINES Repository is the persistent store for assurance case outlines and their derived internal forms. It supports continuity across sessions and enables traceability from outline evolution to instantiated cases and published CAP artifacts.

In addition to source ACO files, the repository may store parsed/canonical internal representations, transformation results, and metadata (including provenance and tool versions), together with any Workbench state required to reproduce assessments, diagnostics, and views. OUTLINES is distinct from CASES (instantiated cases) and EVIDENCE (supporting artifacts), reflecting the lifecycle separation between outline development, instantiation, and evidence management (MedSecurance Project, 2025a).

4.12 Certification Assurance Package (CAP)

The certification assurance package is a collection of artefacts, rendered from data in the CASES and EVIDENCE Repositories, into a form that is suitable for presentation as part of an application for certification under a particular certification regime.

4.12.1 Tailoring the CAP to certification requirements

Figure 9 illustrates how domain-specific certification regime requirements can be used to control the evidence generation processes, the operation of the ETB, the structure of the assurance case and the final rendered content of the CAP for a specific certification regime or project.

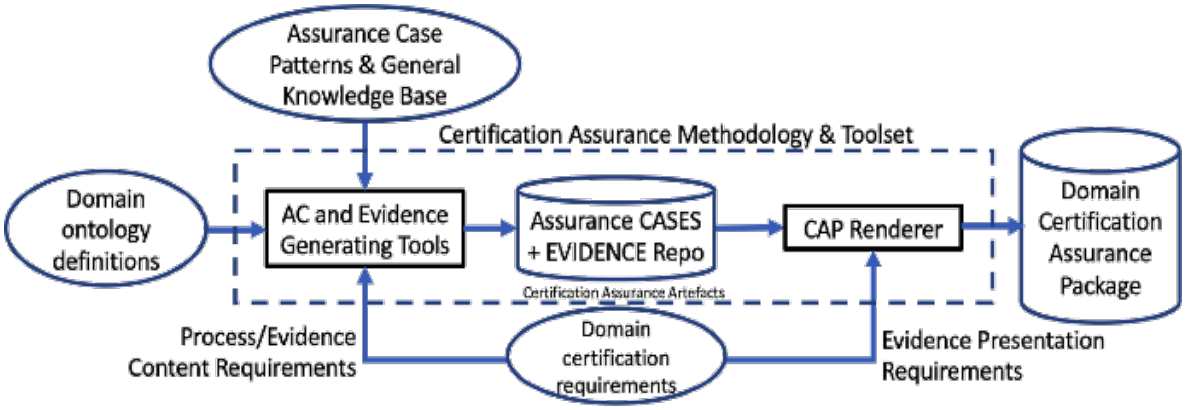


Figure 9: Tailoring the CAP to certification requirements

4.12.2 Organising multiple certification projects

Currently the staging area for certification assurance artefacts is a single directory named "CAP". Since the export operations of the O-ETB enable the file and directory names for the exported items to be specified, a user-chosen naming convention can assist in managing the contents of the CAP directory. Notwithstanding this possibility it would be useful to provide organisation within the CAP directory as a standard feature of O-ETB and this is anticipated as a future extension. This extension would include commands and APIs that provide the ability to register a certification project. The certification registry could be maintained as persistent database at the top-level of the CAP directory. Then, using the certification project identifier as a key, the maintenance of the process and presentation requirements of the certification could be stored in the Knowledge Base and the suitably presented artefacts could be organized in the CAP by subdirectories named for the project identifiers.

4.12.3 Text file artefacts

Assurance cases can be exported in textual form in a .txt file of specified name.

4.12.4 HTML directory artefacts

Assurance cases can be exported in graphical form as a collection of HTML files in a specified directory name. An ordinary web browser may be used to navigate the AC.

5 Assurance Tool Examples

Several examples of the use of the Assurance Tool have been encoded as procs in the file procs.pl. To gain familiarity with interactive use of the tool the commands defining these procs may be entered interactively one-at-a-time. When this is done each command must terminate with a ".".

No Assurance Server examples are presented. Further implementation of the Assurance Server has been deferred unless a clear need for it arises in the MedSecurance Project. The current workflow of the MedSecurance methodology involves interactive and scripted operation of the Assurance tools and does not require programmatic invocation of the O-ETB functions. All automatically sequenced use of the functions can be currently be adequately accomplished by predefined command procs and scripts.

5.1 Example 1 – O-ETB Tool instantiating a simple pattern

This example is encoded as the person_inst procedure in procs_etb.pl as shown in Table 4.

Table 4: Table Example 1 – instantiating a simple pattern and exporting the result

```
proc(person_inst, [
instantiate_pattern(person, ['Marius', 'Programming'], person_examp),
ac_export(cap_person,txt),
detach_case
]).
```

In this procedure the command instantiate_pattern causes the pattern person to be instantiated with the arguments given in the list and will be stored in the CASES Repository with the case identifier person_examp.

Table 5 shows a transcript of listing and executing the person_inst procedure with the optional verbose argument. It instantiates the pattern "person" and creates the assurance case "person_examp" in the CASES Repository.

Table 5: Table 5 Running the person_inst procedure

```
etb> show_proc(person_inst).
:- dynamic procs:proc/2.
procs:proc(person_inst,
[ instantiate_pattern(person, ['Marius', 'Programming'], person_examp),
export_case(cap_person, txt),
detach_case
]).
etb> proc(person_inst,verbose).
> instantiate_pattern(person,['Marius','Programming'],person_examp).
*** instantiating pattern person ... done.
goal g0_1 : {Marius} is sufficiently trustworthy
goal g1_2 : {Marius} has the necessary attributes to perform {Programming}
strategy : argument over attributes
goal g11_3 : {Marius} has sufficient {capability} to perform {Programming}
evidence certificate : {Marius} {capability} for {Programming}
ongoing reference -> 10001
goal g11_4 : {Marius} has sufficient {experience} to perform {Programming}
evidence certificate : {Marius} {experience} for {Programming}
ongoing reference -> 10002
goal g2_5 : {Marius} has sufficient level of supervision and review to successfully perform
{Programming}
> export_case(cap_person,txt).
> detach_case.
etb>
```

As part of the verbose setting the text of the resulting instantiated assurance case pattern is also sent to the console in text form. It is important to execute the command "detach_case" after working with a new AC.

In the procedure of this example the command export_case causes the assurance case in the working cache to be exported to the CAP directory as text in the file cap_person.txt, which you can inspect for yourself.

5.2 Example 2 – Assurance Tool exporting an assurance case

The next example is the procedure "person_exp" (for person export) in Table 6 shows operations on the already existing instantiated pattern stored in the CASES Repository under the identifier "person_examp".

This example is encoded as the person_exp procedure in procs_etb.pl as shown in Table 6.

Table 6: Table Example 2 – exporting an assurance case as HTML

```
proc(person_exp, [
update,
attach_case(person_examp),
ac_export(cap_person,html),
detach_case
]).
```

Table 7 shows a transcript that lists and executes the person_exp procedure with the optional verbose argument.

Table 7: Table 7 Running the person_exp procedure

```
etb> show_proc(person_exp).
:- dynamic procs:proc/2.
procs:proc(person_exp,
[ update,
attach_case(person_examp),
export_case(cap_person, html),
detach_case
]).
etb> proc(person_exp,verbose).
> update.
*** updating evidence repository ... done.
> attach_case(person_examp).
> export_case(cap_person,html).
> detach_case.
etb>
```

The procedure first performs a precautionary call to the update command, which will cause any pending updates to the repositories to be synchronized.

After performing any pending updates the procedure attaches the previously existing case ("person_examp") and then exports it as a directory named CAP/cap_person containing the HTML representation components.

An ordinary browser can be used to open the CAP/cap_person/index.html file in the new directory. By clicking on the different elements listed on the left (in this simple example there is only one element "person_g0_1") you can see the graphical representation as shown in Figure 10 shows the instantiation of the person pattern.

cap_person

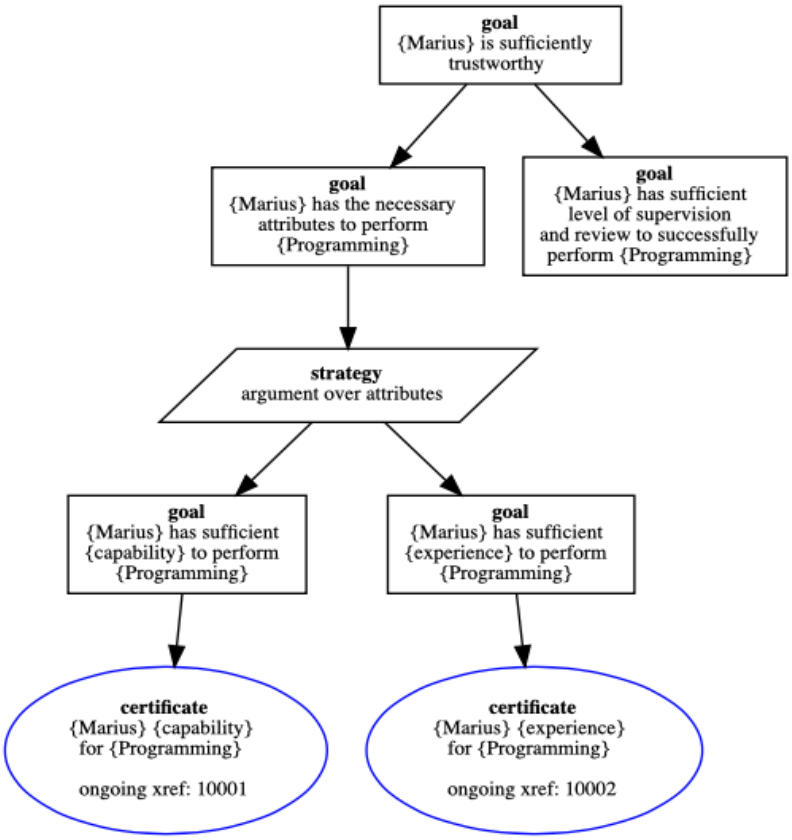
person_g0_1

person_g0_1

Pattern 'person'

- P : person:participant = 'Marius'
- A : person:activity = 'Programming'

GSN Representation



Error Log

Figure 10: Argument pattern instantiation for a person

5.3 Example 3 – Assurance Tool loading a model and instantiating a pattern list

This example is encoded as the system_inst procedure in procs_etb.pl as shown in Table 8.

Table 8: Table Example 3 – instantiating a pattern list from a model and exporting as HTML

```
proc(system_inst, [
set_v(ModelId, '1.0'),
set_v(CaseId, system_examp),
load_model_v(ModelId, Policy, Platform, _Configuration),
set_v(AC, [
'foundational_plane'-[Platform],
'operational_plane'-[Policy],
'person'-['Alice', 'AC Patterns Definition'],
'person'-['Bob', 'ETB Development]
]),
instantiate_pattern_list(AC, CaseId),
ac_export(CaseId, html),
detach_case
]).
```

This procedure uses the logical variables ModelId, CaseId, Policy, Platform, _Configuration and AC to represent values in the procedure (see Section 2.2.3 for notes on syntactic conventions). The transcript of the execution of this procedure is rather lengthy with the verbose option and is not reproduced here. However, it is instructive to use a browser to view the exported HTML in CAP/system_examp directory as this represents a more extensive example.

Table 9: Table Abbreviations

6 References

1. Medical Device Coordination Group (MDCG), https://ec.europa.eu/transparency/expert-groups-register/screen/expert-groups/consult?lang=en&groupID=3565

2. Interfaces and Workflow Definition for AM-ETB, CITADEL Project D5.1, May 2017.

3. AM-ETB Tool Bus for Tool Integration and Assurance, CITADEL Project D5.2, May 2018.

4. CITADEL project, http://www.citadel-project.org, 2019.

5. ISO/TC 215 CD 81001-2-1 Assurance cases, draft for comment, 18 December 2023.

6. N. Shankar, et al, DesCert: Design for Certification, SRI International, arXiv:2203.15178, March 2022.

7. Goal Structuring Notation Community Standard Version 3, The Assurance Case Working Group (ACWG), SCSC-141C, May 2021.

8. Y. Matsuno, Design and Implementation of GSN Patterns: A Step toward Assurance Case Language, In-formation Processing Society of Japan, IPSJ Transactions on Programming 7(2):1-10, 2014.

9. S. Gregory and M. Paschali, A Prolog-based language for workflow programming, University of Bristol, technical report CSTR-07-001, February 2007; and Proceedings of the 9th International Conference on Coordination Models and Languages, June 2007.

10. Object Management Group (OMG), Structured Assurance Case Metamodel (SACM), Version 2.0, 2018.

11. Medical Device Coordination Group, "MDCG 2019-11 Guidance on Qualification and Classification of Software in Regulation (EU) 2017/745 – MDR and Regulation (EU) 2017/746 – IVDR" Oct. 2019.

12. Medical Device Coordination Group, "MDCG 2019-16 Rev.1 Guidance on Cybersecurity for medical devices," 2020.

13. MedSec-Sec Control Taxonomy v2, https://github.com/MedSecurance/Internal/tree/main/WPs/WP3/Ontology Sec_Control_Taxonomy_Version 0.2.xlsx .

14. J. Rushby, An Evidential Tool Bus. In Formal Methods and Software Engineering, 7th International Conference on Formal Engineering Methods (ICFEM), 2005.

15. N. Selviandro, Assurance case pattern using SACM notation. In 2021 9th International Conference on Information and Communication Technology (ICoICT), pp. 494-499. IEEE, 2021.

16. R. Wei, T. P. Kelly, X. Dai, S. Zhao, and R. Hawkins, Model based system assurance using the structured assurance case metamodel. Journal of Systems and Software, 154:211-233, 2019.

17. T. A. Beyene and Harald Ruess, Evidential and Continuous Integration of Software Verification Tools. Proceedings 22nd International Symposium, FM, FLoC, 2018.

7 Abbreviations

Table 9 lists the meanings of acronyms and abbreviations used in this User Manual. A comprehensive collection of abbreviations and terms used in the broader context of the use of O-ETB is referenced in Annex E: Assurance Glossary.

Table 10: Table 10 O-ETB v1.1.3 directory tree

AC	Assurance Case
A-G	Assume-Guarantee
CR	CASES Repository
CAP	Certification Assurance Package
CWE	Core Workflow Engine
ER	EVIDENCE Repository
ETB	Evidential Tool Bus
GSN	Goal Structuring Notation
KB	Knowledge Base (O-ETB component)
LTL	Linear Temporal Logic
MS	MedSecurance
O-ETB	Open ETB
OMG	Object Management Group
SACM	Structured Assurance Case Metamodel – a standard from OMG for expressing assurance cases and evidence
TA	Tool Agent components

8 Annex A: O-ETB Configuration Identification

8.1 File identification

The directory tree of the O-ETB v1.1.3 source repository and distribution is given in Table 8. A description of the contents and purposes of these directories and files is covered in Section 2.1 and a description of the contents of the files containing source code modules in Section 8.2.

Table 11: Table 11 O-ETB v1.1.3 source code modules

./REPOSITORY/EVIDENCE	
./REPOSITORY/EVIDENCE/certificate	
./REPOSITORY/EVIDENCE/certificate/README.md	
./REPOSITORY/EVIDENCE/axiom	
./REPOSITORY/EVIDENCE/axiom/README.md	
./REPOSITORY/EVIDENCE/README.md	
./REPOSITORY/EVIDENCE/ocra	
./REPOSITORY/EVIDENCE/ocra/README.md	
./REPOSITORY/EVIDENCE/unknown	
./REPOSITORY/EVIDENCE/unknown/README.md	
./REPOSITORY/EVIDENCE/repository.pl	./src/stringutil.pl
./REPOSITORY/EVIDENCE/ichecker	./src/models_api/configuration.pl
./REPOSITORY/EVIDENCE/ichecker/README.md	./src/models_api/policy.pl
./REPOSITORY/CASES	./src/models_api/model.pl
./REPOSITORY/CASES/README.md	./src/models_api/platform.pl
./REPOSITORY/README.md	./src/models_api/common.pl
./LICENSE	./src/patterns.pl
./KB/PATTERNS	./src/procs_etb.pl
./KB/PATTERNS/patterns_ISO_81001.pl	./src/makefile
./KB/PATTERNS/patterns_MILS.pl	./src/etb_server.pl
./KB/MODELS/1.0	./src/etb.pl
./KB/MODELS/1.0/configuration.pl	./src/agents/ocra_agent.pl
./KB/MODELS/1.0/system-invariants.inv	./src/agents/axiom_agent.pl
./KB/MODELS/1.0/policy.pl	./src/agents/ichecker_agent.pl
./KB/MODELS/1.0/platform.pl	./src/agents/certificate_agent.pl
./KB/MODELS/1.0/properties	./src/export.pl
./KB/MODELS/1.0/properties/p1.pro	./src/assurance.pl
./KB/MODELS/1.0/properties/p2.pro	./src/instantiate.pl
./KB/MODELS/1.0/system.bip	./src/agent.pl
./KB/README.md	./src/command_etb.pl
./KB/WORKFLOWS	./src/demo.pl
./docs/README-O-ETB-v1	./src/kb.pl
./docs/README-AM-ETB-v3	./src/com/command.pl
./docs/O-ETB-v1.1.0-User-Manual.pdf	./src/com/ui.pl
./README.md	./src/com/param.pl
./CAP/README.md	./src/com/test.pl
./BUILD/etb	./src/com/README.md
./BUILD/README.md	./src/com/procs.pl
./TEST/README.md	./src/evidence.pl

8.2 Module identification

The modules contained in the O-ETB v1.1.3 are listed in Table 9 along with, for each module, a brief description, modules it uses, interfaces it provides, and data structures it creates or uses.

Table 12: Table Evidence artefacts and categories

Module	Description	Modules Used	Interfaces Provided	Data Structures
agent	Invokes agent for evidence category; maintains status	evidence	Evidence_validate	Evidence repository
agents	Directory containing separate agent module per evidence category	axiom_agent certificate_agent ichecker_agent ocra_agent	Axiom_validate certificate_validate ichecker_validate ocra_validate	
assurance	Manages AC repository	persistency	Reset_assurance_repository init_assurance_repository attach_assurance_repository insert_ac_instance insert_ac_instance_id_args ac_instance ac_instance_id_args	Persistent data: ac_instance ac_instance_id_args ac_instance_id_counter
command	Command line interpreter (CLI) with command procedures and debug assist	param ui	Enumerated commands	Command syntax, argument specs, help text, operational semantics per command
command_etb	ETB-specific commands; included in command module	etb	Enumerated commands	

Module	Description	Modules Used	Interfaces Provided	Data Structures
etb	Entry point for interactive tool, process arguments, initialize etb, self-test, start CLI	param command test procs models_api patterns assurance evidence agents instantiate export	Command line args Etb etb_server etb_reset	
etb_server	Entry point for server to run in background, initialize, self-test and start server	etb param models_api patterns assurance evidence agents instantiate export etb_server_api	Server invocation	
evidence	Manages evidence repository	persistency	reset_evidence_repository attach_evidence_repository ac_evidence insert_ac_evidence update_ac_evidence update_ongoing	Persistent data: ac_evidence ac_evidence_counter
export	Build and maintain the certification assurance package (CAP) documents	assurance evidence stringutil	Reset_CAP ac_export ac_format	Assurance case and evidence repositories
instantiate	Instantiate a parameter-ized AC pattern using actual parameters	models_api patterns assurance evidence agent	instantiate_pattern instantiate_pattern_list	Dynamic data: Ac_pattern_pending Ac_pattern_running

Module	Description	Modules Used	Interfaces Provided	Data Structures
kb	Knowledge base operations (placeholder)	patterns assurance evidence	load_specification_from_file kb_reset	Knowledge base: models, patterns, workflows, . . .
models_api	Directory containing modules that provide access to models in the knowledge base	common configuration model platform policy	Interfaces provided by the used modules	Static data: models in the MODELS section of the Knowledge Base (KB)
param	Global parameter file; file and directory names, switches, settings, etc	N/A	Various parameter values and interfaces to construct complex parameters from primitive parameter values	Self-contained data
patterns	Provides interface to multiple AC pattern files	Pattern files specified as param:pattern_files	Pattern library init ac_pattern	Patterns contained in the various pattern files
procs	Stored CLI procedures	CLI commands	General defined procedures by name	Self-contained proc definitions
procs_etb	ETB-specific CLI procs	CLI commands	ETB-specific defined procedures by name	Self-contained proc definitions
stringutil	Utility functions	N/A	Special-purpose string manipulation operations	N/A
test	Utility functions	Self-test modules declared in param	Operations to invoke individual tests or predefined sets of tests	Test cases defined in individual self-test modules
ui	Utility functions	N/A	Basic terminal interaction ops	N/A

9 Annex B: The O-ETB Knowledge Base

9.1 Populating the O-ETB knowledge base

9.1.1 Approach

O-ETB comprises a set of mechanisms and associated configuration information that plays a vital role in the function and application of O-ETB's mechanisms. In the following we present a roadmap for the use of O-ETB in the context of the project's declared use cases, since, as a practical matter, the O-ETB configuration information to be developed first will be those that address the use cases. Some of the configuration artefacts thus developed may be generally applicable, others may be capable of being subsequently generalised and some will be specific to the use case.

The manifest capabilities of O-ETB are dependent upon, and will increase with, the growth and maturity of its knowledge base, enabling it to progress from crawling, to walking, to running as the experience of its users is gained and encoded with a combination of generality and diversity. The strategy underlying the roadmap recognizes that the O-ETB configuration information must be bootstrapped through the initial efforts to apply the MS methodology to the MS UCs.

The O-ETB adoption roadmap, and maturity of the knowledge base, is envisioned as three evolutionary stages:

- Stage I – Near Term (M18-M24) – incorporation of MS methodology and tools into O-ETB configuration information, import of system models and development of appropriate AC patterns in conjunction with development of MS UCs;
- Stage II – Medium Term (M25-M36) – automation of UC AC (re-)construction and CAP rendering through completion of model import and AC patterns;
- Stage III – Post-project (M36+) – delivered MS KB, including contributions from all UCs, serves as basis for future use of O-ETB on MS-compliant developments and other projects, with continuing development of O-ETB capabilities.

These stages are detailed in the following.

9.1.2 Stage I – initial UC adoption of MS methodology/tools

In this stage we assume availability of the O-ETB Assurance Tool core functionality. We do not assume availability of automated import procedures for system model and property specifications, pre-prepared AC patterns, certification requirements specifications, nor O-ETB tool agents for all tools involved in the MS methodology. Rather, in this stage, the AC patterns and model/property specification are developed manually, and the evidential tools are run manually.

9.1.2.1 Prerequisites

- MS methodology and tool descriptions
- Deployable and usable tools, including O-ETB Milestone #1 release
- Defined UCs

9.1.2.2 Activities

- Develop models for the use case and specify properties using the modeling tools.
- Manual application of MS methodology and execution of tools against UC requirements, claims, models, properties, designs and implementations.
- Successful completion of all analyses and assurance activities required by the methodology and applicable standards.
- Use the analysis tools to verify correctness, completeness and validity of the models and properties.
- Develop AC patterns that link the overall goals/claims to the analyses performed.
- Encoding of a AC pattern including all claims and assurance activities carried out according the methodology.
- Implement tool agents as needed to invoke all tools involved in evidence for the instantiated AC.
- Implement any human interface agents for evidence categories for manual assurance activities (those for which there is currently no automated evidence generation procedure).

9.1.2.3 Results

- UC system models capable of being processed by the modeling tools.
- UC system properties capable of being verified by MS and other 3rd party analysis tools.
- Abilty of O-ETB to prompt human users to perform manual assurance activities.
- Claims about the system, relative to its fitness for purpose, that can be substantiated based on the results of assurance activities including the processing of specification artefacts by analysis tools, requirements tracing, testing, reviews, and audits.

9.1.3 Stage II – UC assurance automation for maintenance and certification

In this stage we bootstrap the O-ETB KB by developing its content to make O-ETB able to automate AC construction, evidence generation and certification artefact creation. We develop the model import procedures, AC patterns, certification requirements, and tool agents for all tools involved in the MS methodology and used by the MS UCs. The goal is to automate the assurance process so that it can economically and consistently be rerun for system maintenance and diverse certifications. The effort is devoted primarily to enabling O-ETB to automate the MS UC assurance process as much as practical.

9.1.3.1 Prerequisites

- Understanding of the claims to be established and the rationale that links evidential results of particular evidence categories to support the claims.
- Understanding of how to represent AC goal-argument-evidence structures in the O-ETB AC pattern language.
- Understanding of how the tools are used to produce supporting evidence instances of their corresponding evidence categories.
- Understanding of how tool run results are recorded in records in the EVIDENCE Repository.
- Successful manual application of the MS methodology and manual use of the MS tools.
- Understanding of the process, content and evidence presentation requirements of target certifications.

9.1.3.2 Activities

- Develop the automated model/property KB import functions.
- Encode the steps of the MS methodology as AC patterns in the O-ETB pattern language.
- Implement any tool agents that do not yet exist for the evidential tools used in the UC.
- Implement any export functions that do not yet exist for the artefacts needed for target certification.

9.1.3.3 Results

- The ability to automatically extract and import information from system models and properties into the O-ETB KB.
- AC patterns needed for the MS UCs in the KB and the ability to automatically instantiate those patterns and validate the supporting evidence.
- Tool agents for all tools used in the assurance of the UC and referenced by evidence category at evidence nodes in the AC provide the ability for the O-ETB core workflow to execute external tools and to update the EVIDENCE Repository with the result.
- Ability to generate certification artefacts corresponding to a specific certification project and placed in the CAP.

9.1.4 Stage III – dissemination of the approach & ongoing O-ETB development

This stage, after the MedSecurance Project, is enabled by the previous stages completed within the MS project and provides the material for future use and further development of the O-ETB Assurance Tool.

9.1.4.1 Prerequisites

- The results of Stage II

9.1.4.2 Activities

- Development of extensions to and generalizations of AC patterns reaulting from Stage II to be added to the KB.
- Development of tool agents for tools for new evidence categories or alternative tools for existing categories.
- Extension of O-ETB KB with additional expert knowledge in the form of facts and rules that can be referenced as parameters from future AC patterns.
- Implementation of other previously identified but as yet unimplemented O-ETB features (e.g. MAY requirements).

9.1.4.3 Results

- Expanded and enhanced O-ETB KB, including AC patterns, evidence categories and certification strategies.

- Increased O-ETB functional capabilities, including the ability to use additional types of system model and property information, increased efficiency and parallelism of assurance activities, and enhanced and more diverse export capabilities for improved presentation of certification evidence.
- Ability for O-ETB to be applied to an ever broader range of application domains.

9.1.5 Summary

The O-ETB comprises a mechanism and a configuration. The configuration, which tailors the mechanism to particular methodologies, assurance requirements, application domains and environments, is comprised of a knowledge base (KB) and a collection of tool agents that interface the O-ETB with external tools. The KB is used to encode and preserve experience gained during the development of effective assurance strategies and capturing the ability to present convincing assurance evidence to third party evaluators.

The initial development of the MedSecurance Use Cases, while employing MedSecurance methodology and tools, provides the opportunity to bootstrap the O-ETB from being like an (empty) computer (mechanism) to being a computer with a useful program (mechanism + configuration). The development of that configuration KB for IoMT systems built using the MedSecurance approach will be a key product of the Assurance Tool effort in the MedSecurance Project.

9.2 Model information

9.2.1 Architectural model

A key principle [Kelly] applied in O-ETB is that of structuring the assurance case according to the architecture of the system to be assured. This is reflected in the argument pattern language as constructs to iterate over architectural entities. The model-related terminology currently utilized in O-ETB is based on an architectural meta-model developed in connection with the MILS architectural approach [Boettcher, et al] as manifest in the European Commission D-MILS and CITADEL projects.

The meta-model's terminology includes some distinct terms of art such as the "planes" of a system: "foundational plane", "operational plane", "configuration plane", "monitoring plane", "adaptation plane" and "certification assurance plane". Despite the distinctness of these terms, familiar terms and architectural constructs can be mapped to the meta-model relatively easily. However, since the meta-model is intended to encompass distributed, dynamically reconfigurable and adaptive systems that are capable of just-in-time runtime certification, not all of the planes of the meta-model are embodied in non-distributed or non-adaptive concrete systems. Furthermore, the certification assurance plane is a design and implementation time activity and not a runtime service for systems that do not require runtime recertification after adaptation.

Of ubiquitous interest is the foundational plane, more commonly referred to as the "platform". It includes the computer processor(s), memories, storage devices, network devices, sensors and actuators, their device drivers and the operating system(s) running on the processors. In some cases the platform includes higher-level middleware that provides communication and coordination services. Typically

the platform is a distributed system where each participating computer is referred to as a “node” connected to other nodes by the network. Nodes can be autonomous or coordinated by the platform software.

The applications, possibly multiple interacting applications, reside in the operational plane (or planes). The term “application space” might be more natural. Thus we have adapted the meta-model terms “foundational plane” = “platform” and “operational plane” = “application space” for our present purposes.

Both the platform and the application space may each be represented as composed of distinct but interacting components, commonly represented in component diagrams. The components of the hardware platform benefit from physical isolation and physical/electrical interfaces, and in turn can execute software components and provide logical isolation and discrete communication as a service to other software component architectures. The software platform components benefit from logical isolation and controlled interactions, via typed ports, enforced by the hardware/OS platform. In this way properties of components and contracts among components can be specified and used for verification of compositions.

The model information as needed by the O-ETB is expressed in the terms of the meta-model described above but the translation can be made from other convenient and more familiar terms on import of the model information. The needed architectural model information be retrieved from the MedSecurance modelling tools in a mutually agreed common exchange format which can then be subject to a straightforward translation and import into the KB as facts in the terminology and representation of the meta-model.

Informally, the terms of the meta-model are (we further use the term “policy architecture” or PA to signify a specific instance of the meta-model):

- Nodes (of the PA)
- Modes (of a Subject)
- Subjects (of the PA)
- Subject-Subject Flows (of the PA)
- Transitions (of a Subject)

Each of these may be iterated over in an AC pattern. Further, there can be an iteration over a specified list of constant values. Please refer to the expanded description of the pattern instantiation procedure in Section 3.3.2.3 of D3.1.

9.2.2 Properties

Properties are expressed in Linear Temporal Logic (LTL).

9.3 Argument Patterns

Argument patterns are expressed in the notation introduced in Section 2.3 of this document and in Section 3.3.2.3 of D3.1.

9.4 Workflow information

9.4.1 Tool flows

Tool flows are currently determined by the defined assurance case argument patterns and the semantics of the instantiation procedure which provides a deterministic sequence of visiting the elements of the pattern.

9.4.2 Workflow programs

This refers to a capability resulting from an envisioned future inclusion of a workflow programming language in the O-ETB implementation.

9.5 Certification requirements

9.5.1 Content/process requirements

A certification regime, a regulation, or a standard may levy prescriptive requirements for specific assurance activities and/or specific content to be presented as part of the technical data (resulting from prescribed processes) submitted in support of an application for certification.

9.5.2 Goal-based requirements

A certification regime, a regulation, or a standard may establish goals that must be achieved by the certification applicant rather than prescribed actions that must be shown to have been done.

9.5.3 Presentation requirements

A certification regime may establish a preferred or required form for the presentation of evidence in support of a certification application.

9.6 Evidence-related knowledge

9.6.1 Evidence categories and artefact kinds

An ***evidence category*** associates evidence in a specific form with one or more activities or tools that generate evidence artefacts of that category.

An ***artefact kind*** is a collection of one or more evidence categories that are used in a similar role in an overall assurance strategy.

Table 13: Table Pattern instantiation correctness tests

Artefact kind ***Evidence Category***	***Description***	***Form***	***Activity/Tool***
Specification artefacts			
Architecture specification		Id of system components and connections by ports	
Invariant specification		State formula	
Property specification		LTL formula	
Contract specification	A relationship among components, pairwise or in groups, that defines interdependencies	Component assume-guarantee spec	OCRA
Verification artefacts			
Invariant verification			
Property verification			
Safety analysis artefacts			
Fault tree			
FMEA table			
Validation artefacts			
Contract consistency			
Contract refinement			
Configuration artefacts			
Resource allocation		Configuration file <format>	
Performance artefacts			
Performance analysis			
Standards conformance artifacts			
Compliance sheet			
Conformance certificate			

9.6.2 Evidential tools

Each evidence category is linked to an activity and/or tool that generates that kind of evidence. The invocation of a tool can be automated as part of instantiating an assurance case pattern by creating a

tool agent that is callable by the O-ETB core engine or the use of the tool can be part of a manual external activity that will require manual update of the evidence record upon its successful completion.

9.7 General knowledge

Facts and rules encoded by experts to be used to refine operation.

10 Annex C: O-ETB Testing

Some groundwork for O-ETB testing has been laid in the current version.

10.1 Functional Tests

The functional testing of the Open Evidential Tool Bus (O-ETB), is accomplished with a combination of two mechanisms:

- a built-in self-test feature, accessible with the selftest command of the 'etb' Assurance Tool, that exercises key functions through internal interfaces, and,
- shell scripts that exercise the RESTful APIs to the same functions.

Both of these methods are used as regression tests during ongoing development and maintenance of O-ETB.

Beyond functional correctness, tests will be developed for volume and load, performance and endurance, and correctness tests for non-functional support as described below.

10.2 Volume and Load Tests

For volume and load tests a simple framework of scripts was constructed. The top level script stress-top first invokes a stress-etbapi script that calls the Assurance Server's interface to establish a session to be used for subsequent assurance case and evidence API tests and then initiates multiple parallel copies of a script that repeatedly makes a series of API calls. The queries made by this script comprise ....

The stress-top script takes a numeric argument that it passes along to each of the sub-scripts to tell them how many iterations of the test cases to perform. The value used for this count should be sufficient to assure that the several sub-scripts are running in parallel. The sub-script checks the results of each query, according to the expected result. Any failure of these comparisons is reported as an "ERROR" output along with the failing test case identifier.

10.3 Performance and endurance tests

To facilitate performance tests, a time command is provided by the 'etb' Policy Tool to allow the user to time the execution of any single 'etb' command, or to time multiple executions of a single command or a command script. In the case that multiple executions are requested, text output from the repeated command executions are sent to a null stream, so as not to overly bias the timing result for the execution of the core functions by the accumulated latency of file or terminal I/O resulting from working through the interactive command interpreter.

This time command is used to measure the time to perform multiple executions of a representative Assurance Tool command:

instantiate(...).

This query is by far the most frequently occurring work of the O-ETB system, others being one-time or infrequent calls, which are not only less frequent but much less computationally demanding. Consequently, this test approach is deemed to provide adequate practical coverage.

correctness of the AC instantiation function. Therefore, a series of tests has been developed to demonstrate this correctness, at the command level, at the HTTP API level, and as self-tests perfomed at startup or on demand. The self-tests are constructed within a test framework organized by source module that facilitates addition of tests for new modules and extension of existing module tests. The correctness test framework compares execution results to known answers for the test cases and report discrepancies.

The 'etb' Assurance Tool has built-in self-tests. These may be run to ensure that expected results are produced after O-ETB installation, or as a regression test during further code development. The self-tests can be run by starting 'etb' normally and entering at the "O-ETB>" prompt the command "selftest" or "regtest", which run all of the currently configured tests.

Self-tests and regression tests use the test framework implemented by the module test.pl. Tests for any module may be created in a similar fashion to the currently implemented tests, by creating a TEST/mod_test.pl file for a corresponding module file mod.pl, which should have an include directive for its corresponding test file. Tests defined as startup tests are run by the selftest command and those defined as regression tests will be run in addition to the defined self-tests by the regtest command.

The file TEST/instantiate_test.pl defines self-test cases with expected results for the pattern instantiation functions in the code file instantiate.pl. A representative sample of currently configured test cases are shown in Table 11.

Table 14: Table Running the system_inst procedure

Test case ID	Test case	Expected result
tc01		Succeed
tc02		Succeed
tc03		Succeed
tc04		Fail
tc05		Fail

10.4 HTTP API correctness tests

This section is a placeholder for tests for a future implementation of the HTTP API.

The HTTP APIs are tested using an environment-provided utility command (curl) that enables HTTP requests to be constructed and sent to a network port on the local or a remote machine (the current tests use a fixed reserved port on the local host). The tests are packaged as shell script files in the TEST/nn-tests directory, with each file name beginning with a number. The directory also contains for each such script a file of the expected result in JSON format. A master script, run-nn-tests.sh, runs each of the individual numbered test scripts in turn and checks the current results against expected results.

Table 12 identifies the automated HTTP API tests currently defined. Every such test is represented by a shell script file with the name corresponding to the Test case ID, and an expected result files (e.g. 01-expected-json) and therefore, as the expected results can be rather lengthy, the expected result is not given here.

11 Annex D: Pattern Instantiation Example

This annex presents the complete transcript (Table 14) of the “system_inst” pattern instantiation Example 3 introduced in Section 5.3. The user should run the example and employ a browser to view the HTML representation of the assurance case generated in this example.

```
etb> show_proc(system_inst).
:- dynamic procs:proc/2.
procs:proc(system_inst,
[ set_v(ModelId, '1.0'),
set_v(CaseId, system_examp),
load_model_v(ModelId,
Policy,
Platform,
_Configuration),
set_v(AC,
[ foundational_plane-[Platform],
operational_plane-[Policy],
person-['Alice', 'AC Patterns Definition'],
person-['Bob', 'ETB Development']
]),
instantiate_pattern_list(AC, CaseId),
export_case(CaseId, html),
detach_case
]).
etb> proc(system_inst,verbose).
> set_v(_1626,'1.0').
> set_v(_1638,system_examp).
> load_model_v('1.0',_1652,_1654,_1656).
*** loading model 1.0 ... done.
> set_v(_1666,[foundational_plane-
[platform(main,[node(node1,[memory(ram,[family(ram),size(16),alignment(4)]),memory(disk,[family(disk),size(1
[policy(main,[subject('S1',[port(p,[],[])],[],[processor(x,[family(cpu),frequency('2GHz'),schedule(70)]),memory(z
['Alice','AC Patterns Definition'],person-['Bob','ETB Development']]).
> instantiate_pattern_list([foundational_plane-
[platform(main,[node(node1,[memory(ram,[family(ram),size(16),alignment(4)]),memory(disk,[family(disk),size(1
[policy(main,[subject('S1',[port(p,[],[])],[],[processor(x,[family(cpu),frequency('2GHz'),schedule(70)]),memory(z
['Alice','AC Patterns Definition'],person-['Bob','ETB Development']],system_examp).
*** instantiating pattern foundational_plane ... done.
*** instantiating pattern foundational_plane_node ... done.
*** instantiating pattern foundational_plane_node ... done.
*** instantiating pattern foundational_plane_nsm ... done.
*** instantiating pattern foundational_plane_tsn ... done.
*** instantiating pattern operational_plane ... done.
*** instantiating pattern operational_plane_interface ... done.
*** instantiating pattern operational_plane_process ... done.
*** instantiating pattern operational_plane_process ... done.
*** instantiating pattern operational_plane_composition ... done.
*** instantiating pattern operational_plane_process_modes_and_transitions ... done.
*** instantiating pattern operational_plane_process_threats ... done.
*** instantiating pattern operational_plane_process_properties ... done.
*** instantiating pattern operational_plane_process_modes_and_transitions ... done.
*** instantiating pattern operational_plane_process_threats ... done.
*** instantiating pattern operational_plane_process_properties ... done.
*** instantiating pattern person ... done.
*** instantiating pattern person ... done.
goal g0_1 : The foundational plane guarantees the local policy is met
- context : The adaptive MILS system model describes the plane
```

```
etb> show_proc(system_inst).
:- dynamic procs:proc/2.
procs:proc(system_inst,
[ set_v(ModelId, '1.0'),
set_v(CaseId, system_examp),
load_model_v(ModelId,
Policy,
Platform,
_Configuration),
set_v(AC,
[ foundational_plane-[Platform],
operational_plane-[Policy],
person-['Alice', 'AC Patterns Definition'],
person-['Bob', 'ETB Development']
]),
instantiate_pattern_list(AC, CaseId),
export_case(CaseId, html),
detach_case
]).
etb> proc(system_inst,verbose).
> set_v(_1626,'1.0').
> set_v(_1638,system_examp).
> load_model_v('1.0',_1652,_1654,_1656).
*** loading model 1.0 ... done.
> set_v(_1666,[foundational_plane-
[platform(main,[node(node1,[memory(ram,[family(ram),size(16),alignment(4)]),memory(disk,[family(disk),size(1
[policy(main,[subject('S1',[port(p,[],[])],[],[processor(x,[family(cpu),frequency('2GHz'),schedule(70)]),memory(z
['Alice','AC Patterns Definition'],person-['Bob','ETB Development']]).
> instantiate_pattern_list([foundational_plane-
[platform(main,[node(node1,[memory(ram,[family(ram),size(16),alignment(4)]),memory(disk,[family(disk),size(1
[policy(main,[subject('S1',[port(p,[],[])],[],[processor(x,[family(cpu),frequency('2GHz'),schedule(70)]),memory(z
['Alice','AC Patterns Definition'],person-['Bob','ETB Development']],system_examp).
*** instantiating pattern foundational_plane ... done.
*** instantiating pattern foundational_plane_node ... done.
*** instantiating pattern foundational_plane_node ... done.
*** instantiating pattern foundational_plane_nsm ... done.
*** instantiating pattern foundational_plane_tsn ... done.
*** instantiating pattern operational_plane ... done.
*** instantiating pattern operational_plane_interface ... done.
*** instantiating pattern operational_plane_process ... done.
*** instantiating pattern operational_plane_process ... done.
*** instantiating pattern operational_plane_composition ... done.
*** instantiating pattern operational_plane_process_modes_and_transitions ... done.
*** instantiating pattern operational_plane_process_threats ... done.
*** instantiating pattern operational_plane_process_properties ... done.
*** instantiating pattern operational_plane_process_modes_and_transitions ... done.
*** instantiating pattern operational_plane_process_threats ... done.
*** instantiating pattern operational_plane_process_properties ... done.
*** instantiating pattern person ... done.
*** instantiating pattern person ... done.
goal g0_1 : The foundational plane guarantees the local policy is met
- context : The adaptive MILS system model describes the plane
```

12 Annex E: Assurance Glossary

A glossary for the assurance methodology and the assurance tool, intended as an aid for reconciliation with the project ontology has been developed.

The docs folder of the O-ETB repository contains a Glossary subfolder which in turn contains elements needed to build the Assurance Glossary document. These include an assurance-refs.bib file containing bibliographic entries for references and a glossary-Assurance.tex file

The Assurance-Glossary folder is the build directory for the document and contains the LaTeX source and a makefile to format the document. Invoking with the simple command “make” will build the output gloss-doc.pdf file. Numerous intermediate work files are created here during the document build process. The makefile provides a “clean” target which will remove all of the workfiles (but not the resulting .pdf) if invoked as “make clean”.